Primeros pasos programando para móviles

Closed · Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

📚 Resumen

- Veremos consideraciones especiales de desarrollar código para dispositivos móviles
- Instalaremos y configuraremos Android Studio. Probaremos la aplicación
- Eliminaremos el texto *Hello World!* y comprobaremos de nuevo la aplicación
- Haremos `git add`, `git commit` y `git push` para subir los cambios al repositorio

Descripción

Desarrollar aplicaciones para uso móvil no es lo mismo que crear aplicaciones para escritorio (*Windows, Linux,...*). Es extremadamente importante que:

- Los recursos (RAM, ancho de red,...) usados por nuestra aplicación sean los mínimos.
- Controlemos el *ciclo de vida* de la aplicación. ¿Qué pasa si llaman por teléfono al usuario mientras nuestra *app* está descargando un vídeo?
- Las aplicaciones sean **responsive**. Deben adaptarse perfectamente a diferentes tamaños de pantalla.
- El *feedback* visual sea inmediato. ¡Nada de tocar la pantalla y que dé la impresión de que la app se ha quedado "pillada"!

Principales plataformas

Android e iOS conforman la pareja vencedora en lo que respecta a sistemas operativos móviles hoy en día.

Usaremos Android como vehículo de aprendizaje pues la cuota de mercado es mayor (86% vs 14%), el precio para la publicación de apps es menor (pago único de 25$ frente a 99$ anuales) y no dependemos de un hardware específico (Xcode sólo funciona en MacOS).

Para comenzar a trabajar en Android, tenemos que haber *bajado* nuestro repositorio. Si ya lo hicisteis, puedes saltarte el siguiente apartado.

Configuración de Git e inicialización del repositorio

Instala Git.

Después, **genera** una clave SSH[1] en tu máquina local. Para ello, abre un terminal (tecla de Windows > cmd.exe) y escribe:

```
ssh-keygen -t rsa
```

Pulsa ENTER varias veces para completar el proceso. Luego, **abre** la carpeta `.ssh` que se habrá creado en tu directorio de usuario[2]. Allí verás dos archivos `id_rsa` e `id_rsa.pub`. **Abre** (con el Bloc de Notas) el archivo `id_rsa.pub` y **copia** (CTRL+C) su contenido.

Después, **navega** a https://raspi y haz login con tu cuenta de usuario. Arriba a la derecha, abre el *popup* de ajustes y **navega** a `Settings`.

A la izquierda verás un panel lateral. Busca `SSH and GPG keys` y **haz click** ahí.

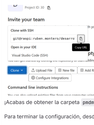
En esa pantalla, verás un recuadro de texto grande. **Pega** (CTRL+V) tu clave pública y **pulsa** en el botón verde `Add key` más abajo.

¡Clave configurada!

Ahora, desde tu terminal (cmd.exe) **clona** (descarga) el repositorio. Para ello, **escribe**:

```
git clone <url_de_tu_repositorio>
```

Donde `<url_de_tu_repositorio>` será la URL que puedes encontrar en https://raspi, yendo a tu proyecto y desde el botón azul de *Clone*:



¡Acabas de obtener la carpeta `pmdm/` (tu repositorio)!

Para terminar la configuración, desde el terminal, **ejecuta**:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "nombre.apellido@fpcoruna.afundacion.org"
```

¡**Sustituye** correctamente `"Nombre Apellido"` y `"nombre.apellido@fpcoruna.afundacion.org"` por tu nombre real y tu correo institucional! (Debes mantener las comillas `" "`).

🏀 La tarea

Desde la página web de Android Studio, **descarga** el instalador `.exe` haciendo click en `Download Android Studio` y **acepta** los términos de licencia[3].

Una vez descargado, **ejecútalo** y procede con las opciones por defecto, hasta que llegues a la ventana donde permite abrir un proyecto existente o crear un proyecto nuevo.

Desde Android Studio, **selecciona** `Open project` y abre la carpeta `android-introduction/` que se encuentra el repositorio local de nuestro ordenador. Puedes darle a `Trust Project`. *Si no has clonado tu repositorio todavía, debes seguir los pasos de más arriba.*

Paciencia 😈

Arrancar un proyecto es lento.

Antes de hacer nada, hay que cerciorarse de que el *banner* amarillo `Gradle Project Sync in Process...` ha desaparecido y abajo no hay barras de progreso indicando tareas en segundo plano. Es posible que Android Studio se encargue de configurar automáticamente la JVM.

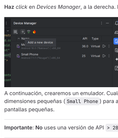
Aunque tarda hasta 5 ó 10 minutos, terminará estabilizándose y veremos a la izquierda nuestra estructura de proyecto:

```
- app/
   |
   |-- manifests/
   |
   |-- java/
   |   |
   |   |-- com.afundacion.fp.library/
   |   |   |
   |   |   |-- MainActivity.java
   |   |
   |   |-- com.afundacion.fp.library/ (androidTest)
   |   |-- com.afundacion.fp.library/ (test)
   |
   |-- res/

- Gradle Scripts/
```

Lanzar la aplicación

Haz click en *Devices Manager*, a la derecha. Pulsa el icono (`+`) para *Create Device*.



A continuación, crearemos un emulador. Cualquier vale, aunque es recomendable usar uno de dimensiones pequeñas (`Small Phone`) para asegurarnos de que nuestra interfaz se adapta a pantallas pequeñas.

Importante: No uses una versión de API `> 28` o tendrás problemas en algunos tests.

Aunque por defecto tendrás una versión del API alta descargada (e.g.: `34`), debes usar una versión del API de Android `<= 28` para que todos los tests funcionen correctamente.

Si usas una más baja, como la `24` ó `25`, es posible que tu emulador funcione más rápido y mejor

Una vez le damos a **Finish**, emulador creado!

Lanza la aplicación con el **botón verde 'Play'** en la barra superior. Elige el emulador recién creado.

¡Y ten paciencia 😅!

Si todo funciona como es debido, aparecerá el emulador a la derecha dentro de Android Studio:

Posibles problemas

- Si la aplicación no llega a lanzarse, es posible que el dispositivo esté *congelado*. Puedes verificarlo si Android Studio responde a tus clicks (e.g.: Abrir menú File) pero el emulador **no** hace **ni caso**. Si te pasa esto:
  1. Cerramos el emulador (icono **X** en la pestaña al lado del nombre)
  2. Vamos a **Dispositivos** (*menú desplegable en la barra de arriba*) > *Device Manager* > *Actions* > Flecha ⏬ (*último icono*) > *Wipe Data*.
  3. **Re-lanzamos** la aplicación con paciencia

Continuemos...

Trabajemos ahora en la aplicación.

Las *interfaces visuales* de las pantallas se definen en archivos XML.

Abre `res > layout > activity_main.xml`. Al hacer doble click en ella, verás:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Elimina las líneas 9–16 asociadas al `TextView`.

Si vuelves a lanzar la aplicación verás que el pequeño texto `Hello World!` ha desaparecido.

¡Enhorabuena! Primera tarea completada.

🏋️ Por último

Desde el terminal de Windows (cmd.exe), nos posicionamos en `android-introduction/` y escribimos:

```
git add *
```

...para marcar los cambios del un nuevo *commit*. Después:

```
git commit -m "Tarea #1: Eliminado TextView hello world en actividad principal"
```

...y por último subimos los cambios al repositorio remoto:

```
git push
```

No está de más visitar la página de GitLab (https://raspi) y verificar que el *commit* se ha subido.

---

1. En realidad son un par de claves: La pública y la privada. ↩
2. Tu directorio de usuario está dentro de la ruta `C:\Users`. *Por ejemplo*, si tu usuario se llama `Pepedepura`, será `C:\Users\Pepedepura`. ↩
3. Estos términos de licencia los impone Google para que usemos su SDK. Básicamente, nos está dando las herramientas necesarias para que nuestras aplicaciones funcionen en dispositivos reales y quiere que, en la medida de lo posible, si ecosistemas de responsabilidad ante los daños que nuestras aplicaciones puedan causar ↩

0 0

Add design

Child items 0

Linked items 0

Activity

All activity · Oldest first

- **Rubén Montero** changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2
- **Rubén Montero** assigned to @pbecerro726 Oct 8, 2026, 12:57 PM GMT+2
- **Pablo Becerro Freire** mentioned in commit 914a4ac97 Oct 8, 2026, 2:49 PM GMT+2

Pablo Becerro Freire closed Oct 13, 2026, 2:51 PM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Un botón

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Añadiremos un botón al archivo de interfaz que editamos en la tarea anterior
- Verificaremos que nuestro botón aparece al lanzar la aplicación
- *(Y como siempre, subiremos nuestros cambios al repositorio)*

Descripción

Los ficheros que describen las *interfaces* (apariencia visual) en Android se escriben en lenguaje XML *(eXtensible Markup Language)*.

Como sabes, en XML la información se estructura mediante *etiquetas*, que son textos delimitados por *menor que* y *mayor que* (`<`, `>`).

Además, la información tiene estructura *jerárquica*, de tal forma que unas *etiquetas* pueden contener otras *etiquetas*. Para ello:

- Se escribe una etiqueta de apertura (e.g.: `<MiEtiqueta>`)
- Se escriben etiquetas *hijas*
- Se escribe la etiqueta de cierre, que es igual pero con un *slash* (`/`) (e.g.: `</MiEtiqueta>`)

En el fichero `activity_main.xml` que editamos en la tarea anterior hay *atributos* y *espacios de nombres* que dificultan la legibilidad, pero puedes apreciar que existe una única etiqueta de tipo:

```
androidx.constraintlayout.widget.ConstraintLayout
```

Un ConstraintLayout es un tipo de *contenedor* Android. En esta tarea, vamos a *añadir* un botón (`<Button>`) dentro del *contenedor*.

La tarea

Edita el archivo `activity_main.xml` que quedó así tras la tarea anterior:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">


</androidx.constraintlayout.widget.ConstraintLayout>
```

Escribe en el espacio en blanco del medio, la *etiqueta de apertura* de `<Button>`:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <Button

</androidx.constraintlayout.widget.ConstraintLayout>
```

Verás que Android intenta autocompletar lo que estás escribiendo. Pulsa **ENTER** para permitirlo:

```
<Button
    android:layout_width=""
    android:layout_height=""
```

Los atributos `android:layout_width` y `android:layout_height` sirven para indicar el ancho y alto del componente.

Escribe `wrap_content`:

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
```

...de esta manera, el ancho y alto del botón se ajustan a su contenido.

¿Qué contenido?

El componente `Button` puede mostrar un texto. **Añádelo** con el atributo `android:text=`, **así**:

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Ver tostada"
```

Para terminar, **cierra** la etiqueta como hemos visto antes:

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Ver tostada">
</Button>
```

Si queremos abreviarlo, podemos escribirlo así:

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Ver tostada"/> <!-- Nótese el slash (/) -->
```

¡Enhorabuena! Has añadido un botón a tu interfaz principal. Ahora puedes lanzar la aplicación con el botón verde de *Play* y, tras esperar a que cargue, verás tu aplicación de nuevo. ¡Ahí está nuestro botón!

Puedes ignorar las advertencias que aparecen en el XML. Nos iremos encargando de ellas en las próximas tareas.

Por último

Haz `git add *` y `git commit -m "Tarea #2, botón añadido"` para crear un *commit*. Sube tus cambios al repositorio con `git push`.

0 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire mentioned in commit 428c9a87 Oct 13, 2026, 3:00 PM GMT+2

Pablo Beceiro Freire closed Oct 13, 2026, 3:01 PM GMT+2

Normal text
Write a comment or drag your files here...
Switch to plain text editing
Make this an internal note
Comment Reopen issue

Assignee Edit
Pablo Beceiro Freire

Labels Edit
None

Milestone Edit
Sprint 1 (expired)

Dates Edit
Start: None
Due: None

Time tracking
Add an estimate or time spent.

2 Participants

ConstraintLayout

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Introduciremos el funcionamiento de `ConstraintLayout` y el uso de *constraints*
- Añadiremos 3 *constraints* al botón

Descripción

El elemento raíz de las interfaces Android con las que trabajaremos, ConstraintLayout, sirve para albergar vistas y posicionarlas de forma sencilla y escalable.

`Constraint`

Son *atributos* de los elementos que especifican su posición *dentro* del `ConstraintLayout`, y tienen una forma como:

```
app:layout_constraintBottom_toBottomOf="parent"
```

(Nótese que el prefijo es `app:` en vez de `android:`)

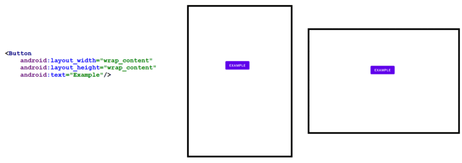
La alerta

En nuestro `<Button>` no hemos especificado ninguna *constraint* y por eso aparece esta advertencia:

> This view is not constrained. It only has designtime position, so it will jump to (0,0) at runtime unless you add the constraints

Es decir, que la razón por la que el botón aparece *arriba a la izquierda* (el origen de coordenadas (0,0)) no es arbitraria, sino un efecto directo de que faltan *constraints*. Podemos imaginar que el `<Button>` está *flotando libremente* porque ninguna *constraint* lo restringe.

En la documentación hay muchos casos de uso bien explicados. Comencemos viendo unas *constraint* básicas:



Como puedes observar, una *constraint* indica:

- ***Qué* (1)** parte del componente se unirá a...
- ***...qué* (2)** parte de otro componente **(3)**.

En el ejemplo, se une el `Top` **(1)** del botón con el `Top` **(2)** del `"parent"`, es decir, el contenedor **(3)**. Si hubiera varios niveles de jerarquía, "*parent*" haría referencia al inmediatamente superior (no necesariamente al `ConstraintLayout` raíz).

```
                (1)   (2)    (3)
app:layout_constraintTop_toTopOf="parent"
```

La siguiente *constraint* une el `Start` **(1)** del botón con el `Start` **(2)** del contenedor **(3)**:

```
                (1)     (2)      (3)
app:layout_constraintStart_toStartOf="parent"
```

`Start` es *izquierda*, con la ventaja de que si nuestra aplicación se ejecuta en un dispositivo con idioma local *Right-To-Left* (como árabe o hebreo), `Start` se posicionará en la *derecha* automáticamente, para ofrecer a esos usuarios su sentido de lectura natural.

Las cuatro dimensiones que podemos usar para *arriba*, *abajo*, *izquierda* y *derecha* respectivamente son:

- `Top`
- `Bottom`
- `Start`
- `End`

La tarea

En `activity_main.xml`, **añade** las *constraints* expuestas anteriormente a tu `<Button>`:

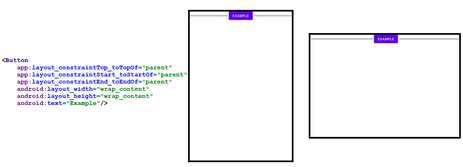
```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Ver tostada"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintStart_toStartOf="parent" >
</Button
```

Si ejecutas la aplicación, verás que el botón aparece *arriba a la izquierda*. Igual que antes, pero ahora no hay ninguna advertencia en el XML indicando que la posición del botón es *ambigua*.

Centrar el botón

Es posible especificar simultáneamente una *constraint* `Start_toStart` y `End_toEnd`.

Dado que el *ancho* está especificado como `wrap_content`, cada *constraint* "tirará" del componente hacia su lado y se centrará[1].



Añade una *constraint* `End_toEnd` para *centrar* tu botón en la parte superior.

Puedes lanzar la aplicación y verificar que el botón aparece centrado arriba.

Por último

Haz `git add *` y `git commit -m "Tarea 3, botón centrado"` para crear un *commit*. Sube tus cambios al repositorio con `git push`.

1. Podrías esperar que el `<Button>` se deformase y se *pegase* a ambos lados, ocupando todo el ancho. Eso puede suceder si el **ancho** es *algo distinto* a `wrap_content`. Lo veremos más adelante. ↩

👍 0 👎 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 15, 2026, 9:28 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment Reopen issue

Assignee Edit
Pablo Beceiro Freire

Labels Edit
None

Milestone Edit
Sprint 1 (expired)

Dates Edit
Start: None
Due: None

Time tracking
Add an estimate or time spent.

2 Participants

Un botón que ocupe todo el ancho

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

🟩🟥 Resumen

- Añadiremos un botón alineado con la parte inferior
- Distinguiremos entre `wrap_content` y `0dp`
- Haremos que el botón ocupe todo el ancho

Descripción

Tal y como hemos visto, hay cuatro posibles valores con los que podemos indicar que nuestras *constraints* ajusten los componentes a:

- Arriba (`Top`)
- Abajo (`Bottom`)
- Derecha (`End`) *(ó `Right`, que es derecha estrictamente, incluso en países Left-to-Right)*
- Izquierda (`Start`) *(ó `Left`, que es izquierda estrictamente, incluso en países Left-to-Right)*

Y, también, los componentes tienen:

- `layout_width` (ancho)
- `layout_height` (alto)

En la tarea anterior hemos aplicado `layout_width="wrap_content"` a un botón. Esto hace que el *ancho* del botón envuelva a su contenido.

Vamos a añadir un botón y usar `layout_width="0dp"`. `0dp` es sinónimo de MATCH_CONSTRAINT, o sea que la *anchura* del botón ya no se ajusta a su contenido, sino que se ajusta a las *constraints*. ¡A continuación lo pondremos en práctica!

Inciso: Sobre `dp`

Cuando indicamos medidas en Android, podemos usar `dp`, `sp` ó `px`.

`px` es la medida en *píxeles*. Como distintas pantallas tienen distintas resoluciones (e.g.: 480x800 vs. 1080x1920), usar `px` es una *mala práctica*. `30px` ocupa *más lienzo* en una pantalla pequeña que en una pantalla grande.

`dp` es la medida en *píxeles de densidad independiente*, es decir, relativos al tamaño de la pantalla.

`sp` es similar a `dp`, pero adecuado para tamaños de textos.

📦 La tarea

En `activity_main.xml`, **añade** una nueva etiqueta `<Button>`. Estará dentro del `<ConstraintLayout>` y será **así**:

```
    <Button
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Lanzar nueva actividad"/>
```

Como ves, la *constraint* `Bottom_toBottomOf="parent"` indica que debe estar pegado abajo.

¡**Lanza** la aplicación y pruébalo! ¿Cómo se ve?

A continuación, **modifica** el valor `"wrap_content"` por `"0dp"` en `layout_width`:

(Las líneas en rojo indican código que hay que borrar, y en verde es código para añadir)

```
    <Button
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
-       android:layout_width="wrap_content"
+       android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Lanzar nueva actividad"/>
```

Lanza de nuevo la aplicación. ¿Cómo ves ahora el botón?

🏆 Por último

Sube tus cambios al repositorio en un nuevo *commit*.

👍 0 👎 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 15, 2026, 11:54 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

TextView, IDs, posición relativa

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

📚 Resumen

- Añadiremos un `TextView`
- Usaremos el atributo `id` en nuestro primer botón
- Posicionaremos el `TextView` *debajo* del primer botón

Descripción

Hasta ahora sólo hemos usado `Button`.

Otro componente esencial en Android es `TextView`. Sirve para mostrar un texto sencillo.

📦 La tarea

Añade un elemento `<TextView>` dentro del `<ConstraintLayout>` de nuestro `activity_main.xml`.
Si escribes `<TextV`, Android Studio autocompletará fácilmente:

```
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

Añade a dicho `<TextView>` el atributo `android:text=` (igual que hemos hecho con los botones) para que indique el texto: *Mermelada no incluida*.

También, **añade** las *constraints* `Start_toStart="parent"` y `End_toEnd="parent"` para **centrar** el texto. En total, te quedará algo así:

```
    <TextView
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Mermelada no incluida"/>
```

¡Ya tenemos el `<TextView>`! Ahora vamos a posicionarlo debajo del primer botón.

Posicionando el texto *debajo* del botón

En primer lugar, **añade** el atributo:

```
    android:id="@+id/toastButton"
```

...**al** `<Button>` de la primera tarea que indicaba `"Ver tostada"`.

Este atributo sirve para dar un identificador único a un componente. Es útil para posicionar elementos de forma relativa en un `ConstraintLayout`. También lo usaremos más adelante, escribiendo código Java.

Ahora, **añade** esta *constraint* a nuestro `<TextView>`:

```
    app:layout_constraintTop_toBottomOf="@id/toastButton"
```

Como puedes ver, se indica que la parte superior del componente (el `Top` del `<TextView>`) debe estar pegada a (`to`) la parte inferior (`Bottom`) del `"@id/toastButton"`.

¡Listo! Si lanzas la *app*, verás el texto *Mermelada no incluida* debajo del primer botón.

🏆 Por último

Sube tus cambios al repositorio en un nuevo *commit*.

👍 0 👎 0 ☺ Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 15, 2026, 11:58 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Ciclo de vida: onCreate

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Introduciremos el concepto de actividad y distinguiremos XML y Java
- Hablaremos brevemente del método `onCreate`
- Mostraremos un *Toast* cuando se crea nuestra actividad

Descripción

La experiencia de usuario con aplicaciones móviles es distinta a las aplicaciones de escritorio. En una *app* móvil no se puede esperar que haya un único punto de entrada. Por ejemplo, si abres la aplicación de Gmail verás tu bandeja de entrada. Pero si seleccionas un PDF desde el explorador de archivos y usas *Compartir con > Gmail*, aparecerá directamente la ventana para redactar un nuevo correo con el archivo adjunto.

Las actividades son el elemento central de la programación Android y facilitan este paradigma.

Debemos entender una *actividad* como una *pantalla*, digamos, un *lienzo* donde nuestra aplicación puede pintar la interfaz de usuario. La mayoría de aplicaciones tienen varias pantallas, por lo que tienen varias actividades.

Hay *tres* piezas que construyen una actividad:

1) Archivo de interfaz XML

Como `activity_main.xml` con el que hemos estado trabajando. Suelen comenzar con `activity_` y luego llevar en `snake_case` el nombre de la actividad.

2) Declaración en el manifiesto XML

El archivo de manifiesto Android es un único archivo XML que tiene cada aplicación. En él se declaran cosas como el *nombre* de la aplicación, el *icono*, o las *actividades*.

Para declarar una actividad, basta con que se incluya en el sitio adecuado (dentro de `<application>`) una etiqueta como:

```
<activity android:name=".ExampleActivity" />
```

Si echas un vistazo al archivo *app > manifests > AndroidManifest.xml* de nuestra aplicación, verás cómo está declarada la (única) actividad de la aplicación.

Observa que tiene un par de cosas adicionales. Entre ellas, `android.intent.category.LAUNCHER` sirve para indicar que es la actividad *inicial*.

3) Clase Java

Asumiendo cierto grado de conocimiento de programación orientada a objetos (POO) diremos que:

- Cada actividad tiene una clase asociada. Esta clase debe *heredar* (ser una subclase) de `Activity` (o `AppCompatActivity`).
- Por ello, el nombre de la clase suele llevar el sufijo `Activity`, e.g.: `MainActivity`.
- Nosotros **no** *instanciamos* nuestra clase. El sistema Android se encarga de ello.
- Nuestra responsabilidad es *sobreescribir* (`@Override`) los **métodos** que responden a eventos.

¿Métodos que responden a eventos?

Sí.

El método principal es `onCreate`. Se trata de un método que **se va a ejecutar una (1) sóla vez** cuando nuestra actividad sea creada.

La tarea

Abre *app > java > com.afundacion.fp.library >* `MainActivity.java`. Puedes ver la declaración de una clase, como habíamos prometido, y, además, hay un método llamado `onCreate`:

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Dentro de este método hay un par de líneas típicas que sirven para inicializar la actividad.

A continuación de esas dos líneas, **añade** una línea que sirva para lanzar un `Toast`[1]:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // Vamos a lanzar un Toast
        Toast.makeText() // ??
    }
```

El método estático `makeText` recibe tres parámetros:

- Un `Context`. Esta clase es esencial en muchos métodos Android. Basta saber que una `Activity` *también* es un `Context`, así que podemos enviar `this`.
- Un texto en `String`. Nosotros usaremos `"¡La tostada del desayuno!"`.
- Una constante que indique si se mostrará un tiempo corto o largo. Sólo vale `Toast.LENGTH_SHORT` y `Toast.LENGTH_LONG`.

Esto devuelve un objeto tipo `Toast` sobre el que podemos invocar `.show()` para que se muestre:

```
Toast oneToast = Toast.makeText(this, "¡La tostada del desayuno!",
Toast.LENGTH_SHORT);
oneToast.show();
```

Por brevedad suele escribirse todo en una línea:

```
Toast.makeText(this, "¡La tostada del desayuno!",
Toast.LENGTH_SHORT).show();
```

¡Adelante! **Añade** esta tostada en ~~nuestro desayuno~~ nuestra aplicación.

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // Vamos a lanzar un Toast
        Toast.makeText(this, "¡La tostada del desayuno!",
Toast.LENGTH_SHORT).show();
    }
```

¡Enhorabuena! Has programado tu primera *tostada*.

Prueba a ejecutar tu *app* en el emulador. ¿Qué ves nada más arranca?

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

1. Un `Toast` es un pequeño elemento visual típico de Android que salta momentáneamente en la pantalla, como una tostada. Es útil para verificar que el código se está ejecutando. ↩

0 0 Add design

Child items 0

Linked items 0

Activity All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 15, 2026, 12:13 PM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Help
Collapse sidebar

Clicar un botón

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Veremos cómo responder a un *click* de un usuario en un botón
- Hablaremos brevemente de interfaces Java
- Mostraremos un *Toast* si el usuario clica en el botón superior

Descripción

Responder al *click* de un usuario en un componente es muy fácil.

Basta con usar un objeto `OnClickListener`:

`OnClickListener`

Es una interfaz Java que declara un único método:

```
public void onClick(View v)
```

Recuerda que las interfaces, en Java, son *firmas de métodos*. Si una clase *tiene* esos métodos, se dice que *conforma* ó *implementa* esa interfaz Java.

Por ejemplo, una interfaz Java *válida* sería:

```
public interface InterfazEjemploNumero {

  public int devolverUnNumero(boolean positivo);
}
```

Como ves, el método acaba en punto y coma (`;`) en vez tener un cuerpo entre llaves (`{ }`). Eso es porque en la interfaz sólo está presente la *firma* del método. Son las *clases* quienes implementan las *interfaces* especificando `implements`, por ejemplo:

```
public class ClaseImplementadora implements InterfazEjemploNumero {

  @Override
  public int devolverUnNumero(boolean positivo) {
    if (positivo) {
      return 10;
    } else {
      return -10;
    }
  }
}
```

La anotación `@Override` sirve para indicar sobreescritura de un método de *superclase* **o** de una *interfaz*. Este doble uso puede despistar a veces, pero no le demos mucha importancia.

La tarea

Añade una nueva clase. Para ello, **haz** *click* derecho en `com.afundacion.fp.library` > *New* > *Java Class*. **Dale** el nombre `TapHandler`.

Añade el siguiente contenido a la clase (te resultará fácil de entender si ya eres familiar con los POJOs):

```
package com.afundacion.fp.library;

import android.content.Context;

public class TapHandler {
    private Context context;

    public TapHandler(Context context) {
        this.context = context;
    }
}
```

Ahora, en `MainActivity.java`, **crea** una nueva instancia de `TapHandler` después del `Toast` de la tarea anterior. **Así**:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toast.makeText(this, "¡La tostada del desayuno!",
Toast.LENGTH_SHORT).show();
        TapHandler myHandler = new TapHandler(this); // Instanciamos un
nuevo TapHandler
    }
```

(Una `Activity` es un `Context`, por eso, podemos pasar `this` en el constructor)

Ahora vamos a identificar y asociar nuestro botón al `TapHandler`.

En `activity_main.xml`, el `<Button>` es una etiqueta.

En Java, será una *variable* de tipo `Button` que no *inicializamos* usando `new`, sino usando `findViewById`.

Adelante, **añade** la siguiente línea:

```
        Button myButton = findViewById(R.id.toastButton);
```

(Como puedes ver, `R.id.toastButton` es el mismo `android:id` que habíamos indicado en el XML)

El penúltimo paso

Vamos a unir nuestro `myHandler` al botón. Eso se hace *invocando* `setOnClickListener` sobre el botón.

Hazlo así:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toast.makeText(this, "¡La tostada del desayuno!",
Toast.LENGTH_SHORT).show();
        TapHandler myHandler = new TapHandler(this);
        Button myButton = findViewById(R.id.toastButton);
        myButton.setOnClickListener(myHandler);
    }
```

Hemos terminado con `MainActivity.java`

No te preocupes por la alerta que aparece en la última línea:

> RequiredType: OnClickListener
>
> ProvidedType: TapHandler

Java está indicando: *"Eh, quería que pasases un `OnClickListener` a este método y me has pasado un `TapHandler` en su lugar"* ¡Lleva razón!

Vamos a...

...hacer que `TapHandler` sea también `OnClickListener`

En `TapHandler.java`, **añade** `implements View.OnClickListener`:

```
import android.content.Context;
import android.view.View;

public class TapHandler implements View.OnClickListener{
    private Context context;

    public TapHandler(Context context) {
        this.context = context;
    }
}
```

Verás que la primera línea aparece subrayada en rojo.

> Class 'TapHandler' must (...) implement abstract method 'onClick(View)'

¡Claro! Tras declarar que se *implementa* una interfaz... ¡Debemos implementarla!

Añade el método `onClick`:

```
public class TapHandler implements View.OnClickListener{
    private Context context;

    public TapHandler(Context context) {
        this.context = context;
    }

    @Override
    public void onClick(View view) {

    }
}
```

Y, para terminar, en el método `onClick`, **haz** que se lance un `Toast`. **Así**:

```
    @Override
    public void onClick(View view) {
        Toast.makeText(this.context, "¡Tostada al aire!",
Toast.LENGTH_LONG).show();
    }
```

¡Enhorabuena! Has conectado tu primer botón en Android.

Ya puedes lanzar `app` y hacer *click* en "Ver tostada", ¿qué ves?

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0
Linked items 0

Activity All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2
Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2
Pablo Beceiro Freire closed Oct 15, 2026, 12:34 PM GMT+2

Normal text
Write a comment or drag your files here...
Switch to plain text editing
Make this an internal note

Clicar un botón sin escribir una clase nueva

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Veremos cómo instanciar una interfaz con una clase *anónima*
- Mostraremos un *Toast* si el usuario clica en el botón inferior

Descripción

Comenzamos la tarea anterior diciendo:

> Responder al *click* de un usuario en un componente es muy fácil.

Sin embargo, tuvimos que escribir una nueva clase, implementar una interfaz... ¡Qué complicado!

Vamos a aprender cómo podemos conseguir lo mismo sin añadir una nueva clase.

La tarea

Como primer paso, en `activity_main.xml`, **otórgale** un `id` al botón inferior:

```
    android:text="Lanzar nueva actividad"
    android:id="@+id/bottomButton"/>
```

Ahora, en `MainActivity.java`, **instancia** el `Button` al final del método `onCreate`:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        // ...
        Button otherButton = findViewById(R.id.bottomButton);
    }
```

Dejemos que Android Studio autocomplete

Tras la línea anterior, **escribe** `otherButton.setOnClickListener()` y **dentro** de los paréntesis, **escribe** `new` . En cuanto insertemos un espacio en blanco, Android Studio nos sugerirá una lista de opciones de autocompletado.

La primera seguramente sea `TapHandler`. En ese caso nos interesa la segunda, `View.OnClickListener{...}`. **Dale** a ENTER:

```
        otherButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {

            }
        });
```

Ya tenemos nuestro deseado `onClick`.

Esto es lo que se conoce como crear una clase *anónima*. Estamos *instanciando* (con `new`) una clase que estamos creando al momento, y por eso no tiene nombre.

Para terminar, **muestra** un `Toast` con el mensaje `"Empezando otra actividad..."` en dicho `onClick`:

```
        otherButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Toast.makeText(this, "Empezando otra actividad...",
Toast.LENGTH_LONG).show();
            }
        });
```

¡Un momento! Android Studio indica un error

Verás un error:

> *Cannot resolve method...*

Se debe a que estamos invocando `makeText` con parámetro incorrectos.

En esa parte del código (dentro de `onClick`), `this` no se refiere a la `Activity`, sino a la clase *anónima* recién creada. Por lo tanto, no es un `Context`. Por lo tanto, no vale.

¿Cómo lo solucionamos?

Añade un atributo de tipo `Context` a `MainActivity` que sea `this`:

```
import android.content.Context;

public class MainActivity extends AppCompatActivity {
    private Context context = this;

    // ...
}
```

Y luego, en la línea problemática, **usa** `context` en lugar de `this`:

```
        otherButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Toast.makeText(context, "Empezando otra actividad...",
Toast.LENGTH_LONG).show();
            }
        });
```

¡Listo! Puedes lanzar tu `app` y verificar que ambos botones muestran distintos `Toast`.

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

Nueva actividad

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Crearemos una nueva actividad
- La iniciaremos si el usuario *clica* en el botón inferior

Descripción

Nuestra aplicación tiene sólo una actividad, `MainActivity`. Vamos a crear otra actividad llamada `MonstersActivity`. Recordemos que una actividad tiene *tres* componentes fundamentales:

- Clase Java
- Interfaz XML
- Declaración en `AndroidManifest.xml`

Vayamos añadiéndolos uno a uno.

La tarea

Clase Java

Añade una nueva clase Java. **Dale** el nombre `MonstersActivity`.

En su contenido, **añade** el siguiente código:

```
package com.afundacion.fp.library;

import androidx.appcompat.app.AppCompatActivity;

public class MonstersActivity extends AppCompatActivity {
}
```

(Como ves, `MonstersActivity` hereda de `AppCompatActivity`. La herencia se declara en Java con la palabra reservada `extends`)

Interfaz XML

En las carpetas de la izquierda, **expande** *res*. **Haz** *click* derecho en *res > layout*. Ahí, **selecciona** *New > Layout Resource File*.

Seguramente ya nos aparezca como **Root element** `androidx.constraintlayout.widget.ConstraintLayout`. Eso está bien.

Ahora, **escribe** `activity_monsters.xml` como **File Name** y dale a OK. Se generará un XML como este:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

</androidx.constraintlayout.widget.ConstraintLayout>
```

Declaración en `AndroidManifest.xml`

Ya hemos visto que en *manifest* > `AndroidManifest.xml` se añaden etiquetas `activity` para declarar nuevas actividades. ¡Hagámoslo!

Añade `<activity android:name=".MonstersActivity"/>` a `AndroidManifest.xml`:

```
    <application
        ...>
        <activity
            android:name=".MainActivity"
            ... >
        </activity>

        <!-- Añadimos una nueva activity debajo de la otra -->
        <activity android:name=".MonstersActivity"/>

    </application>
```

¡Actividad creada y declarada!

Iniciando la actividad con `startActivity`

Vamos a hacer que la *app* navegue a esta nueva actividad cuando el usuario clique el botón "Lanzar nueva actividad."

¿Recuerdas que en `MainActivity.java` ya tenías el siguiente *on click*?

```
        otherButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Toast.makeText(this, "Empezando otra actividad...",
Toast.LENGTH_LONG).show();
            }
        });
```

¡Bien! **Complétalo** añadiendo estas dos líneas para *arrancar* otra actividad.

En primer lugar, **crea** un `Intent` en `MainActivity.java`, como se indica a continuación:

```
        // ...
        // Este código pertenece a onCreate en MainActivity.java
        otherButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Toast.makeText(context, "Empezando otra actividad...",
Toast.LENGTH_LONG).show();
                Intent myIntent = new Intent(context,
MonstersActivity.class);
                context.startActivity(myIntent);
            }
        });
```

(`Intent` representa algo así como una intención. La intención de iniciar la `MonstersActivity`.)

¡Enhorabuena! Has creado y arrancado tu primera actividad desde cero.

Prueba el comportamiento en el emulador. ¿Al hacer *click* ves una nueva pantalla vacía?

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 19, 2026, 9:41 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Help
Collapse sidebar

Un título sencillo

Closed **Issue** created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- En nuestra nueva `MonstersActivity` añadiremos un `TextView`
- Aprenderemos sobre los atributos `margin`, `textSize` y `textColor`

Descripción

Vamos a añadir un texto de encabezado a la nueva actividad.

Para ello, editaremos el archivo de interfaz XML.

`activity_monsters.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

</androidx.constraintlayout.widget.ConstraintLayout>
```

La tarea

Añade a `activity_monsters.xml` un `TextView` con el texto **"Biblioteca de monstruos"**:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Biblioteca de monstruos"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Con *constraints*, **posiciónalo** centrado horizontalmente y ubicado en la parte superior. **Así**:

```
    <TextView
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Biblioteca de monstruos"/>
```

(Android Studio resaltará `app` en rojo. La ayuda nos permitirá autocompletar `xmlns:app="http://schemas.android.com/apk/res-auto"`, que es la definición de espacio de nombres necesaria para que funcione)

Podemos indicar `android:layout_margin{Top|Bottom|Start|End}` para que el componente esté espaciado una cantidad de `dp` en dicha dimensión.

Añade el siguiente `layout_marginTop` para alejarlo un poco del margen superior:

```
        android:layout_marginTop="10dp"
```

Y, a continuación, **añade** los atributos `textSize` y `textColor`:

```
        android:textSize="26sp"
        android:textColor="#040701"
```

Recuerda que para el tamaño de texto debemos usar las unidades `sp`, y para el color indicaremos un código de color HTML.

`onCreate`

Nuestras modificaciones en `activity_monsters.xml` no servirán para nada si no tenemos el código de inicialización apropiado en `MonstersActivity.java`.

Abre `MonstersActivity.java`, y, dentro de la clase, **escribe** `onCrea`... ¡Verás que Android Studio autocompleta!

```
public class MonstersActivity extends AppCompatActivity {

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
    }
}
```

Luego, **añade** `setContentView` para *enlazar* nuestro XML a la actividad. **Así**:

```
public class MonstersActivity extends AppCompatActivity {

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_monsters);
    }
}
```

Prueba ahora el resultado. ¿Qué tal se ve?

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 19, 2026, 10:11 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Ciclo de vida: onStop

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Veremos las posibilidades en el ciclo de vida de una actividad Android
- Implementaremos `onStop` en nuestro `MainActivity`

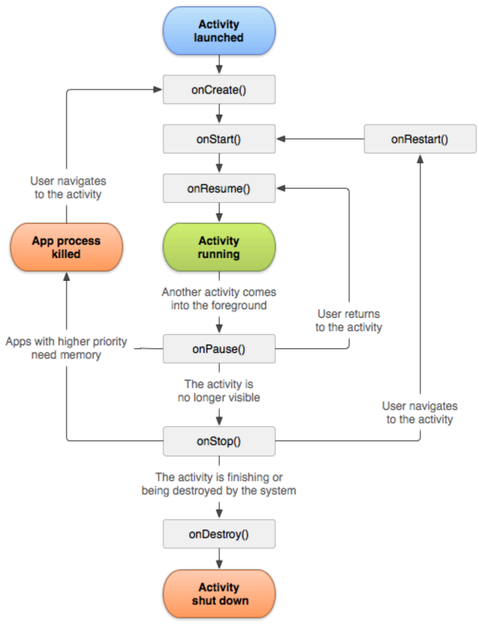
Descripción

El *ciclo de vida* de una actividad Android consiste en los posibles estados que puede atravesar mientras el usuario disfruta de ella.

Inicialmente se creará y entonces `onCreate` será invocado.

¿Qué más puede pasar?

Ciclo de vida



Nosotros vamos a utilizar `onStop`. Sirve para controlar que la *app* ha dejado de estar en primer plano, igual que `onDestroy`. La diferencia es que si hay escasez de memoria RAM, el sistema Android puede matar nuestra aplicacion *sin* invocar `onDestroy.`

La tarea

En `MainActivity.java`, añade un nuevo método *dentro* de la clase. **Teclea**:

```
onSto
```

Verás cómo Android Studio autocompleta por nosotros. **Dale** a ENTER para que eso pase:

```
public class MainActivity extends AppCompatActivity {
    private Context context = this;

    @Override
    protected void onCreate(Bundle savedInstanceState) { /* ... */ }

    @Override
    protected void onStop() {
        super.onStop();
        // ...
    }
}
```

A continuación, *dentro* de `onStop` (después de `super.onStop()`), **lanza** un `Toast` con el texto `"Hasta luego, MainActivity"`.

Una vez lo hayas escrito, **prueba** tu `app` en el emulador.

¿Qué ves si haces click en el botón inferior para empezar la otra actividad?

¿Qué ves si pulsas el botón de *Atrás* un par de veces para salir de la aplicación

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0
Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 19, 2026, 10:17 AM GMT+2

Normal text
Write a comment or drag your files here...
Switch to plain text editing
Make this an internal note
Comment Reopen issue
Help
Collapse sidebar

Imágenes

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Aprenderemos sobre el componente `ImageView`
- Mostraremos una imagen en `MonstersActivity`

Descripción

Hasta ahora hemos usado botones y textos.

Son dos componentes fundamentales en desarrollo móvil, pero a los usuarios también le gusta observar recursos gráficos en las aplicaciones.

`ImageView` es un componente que sirve para mostrar una imagen. Nosotros vamos a usar una imagen que *ya viene* con la aplicación.

¿*Ya viene* con la aplicación?

Sí. En la carpeta `res/` podemos añadir distintos tipos de *resources* (recursos). Algunos, como ya sabes, son archivos XML que describen las interfaces.

Pues bien, dentro de `res/drawables/` es común añadir *imágenes* (`.png` , `.jpg`) que mostraremos dentro de nuestra aplicación.

¡Ojo! No queremos que estas imágenes ocupen demasiado, pues *aumentarán el tamaño de la aplicación*. ¿Alguna vez has ido a bajar una aplicación de la PlayStore y después de ver que pesaba 200Mb has cancelado la descarga?

La tarea

En primer lugar, en `activity_monsters.xml` , **añade** un `android:id` al `TextView` que ya habíamos creado en una tarea anterior:

```
    <TextView
        android:id="@+id/monstersTitle"
```

Luego, **añade** un `ImageView` :

```
    <ImageView
        android:layout_width=""
        android:layout_height=""
```

Es *raro* que se use `wrap_content` con un `ImageView` . Pantallas de diferentes tamaños mostrarán la misma imagen más grande o más pequeña, pues los píxeles de la imagen son los mismos.

En su lugar, suele concebirse el `ImageView` como un *lienzo*. Ajustamos dicho *lienzo* empleando *constraints*. Para ello, **usa** `0dp` como *ancho* y *alto*:

```
    <ImageView
        android:layout_width="0dp"
        android:layout_height="0dp"
```

A continuación, **añade** *constraints* para posicionarlo *debajo* del título, y *pegado* al borde de la pantalla por izquierda, derecha y abajo. **Así**:

```
    <ImageView
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toBottomOf="@id/monstersTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"/>
```

`android:src`

Lo único que nos falta es indicar *qué* imagen debe mostrar. Nosotros vamos a mostrar la imagen de *togemon*.

Añade el siguiente atributo a tu `ImageView` :

```
        android:src="@drawable/togemon"
```

Y... una buena práctica

También es *recomendable* añadir el atributo `contentDescription` a las imágenes.

Este atributo sirve para *describir* una imagen por cuestiones de accesibilidad. Es decir, una persona ciega o con visión reducida usará el *lector de pantalla* y agradecerá disponer de una ayuda para comprender el contenido visual de la imagen.

Para terminar, **añade** este otro atributo a tu `ImageView` :

```
        android:contentDescription="Un Digimon poderoso"
```

¡Listo! Ya hemos mostrado nuestra primera imagen en una aplicación. ¿Qué tal se ve?

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0
Linked items 0

Activity All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 19, 2026, 10:22 AM GMT+2

Normal text
Write a comment or drag your files here...
Switch to plain text editing
Make this an internal note

Barra de navegación inferior

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Añadiremos un archivo XML que representa un *menú*
- Añadiremos una barra de navegación inferior a nuestra `MonstersActivity`
- Vincularemos la barra de navegación a dicho *menú* XML

Descripción

La navegación en Android se basa en actividades. Cada vez que iniciamos una actividad, se añade a una *pila de navegación*. El usuario puede navegar hacia atrás pulsando el botón de *Atrás*.

Pues bien, es bueno no obligar al usuario a *navegar* por demasiadas pantallas. ¡Esto no es la web! En una *app* móvil, preferimos tener todo a golpe de *click*. Cuantas menos interacciones, mejor.

Menús

Los menús son elementos recurrentes en Android. Hay muchas formas de mostrarlos.

Nosotros vamos a añadir un *menú* y permitir al usuario seleccionar sus elementos mediante una *barra inferior*. Al clicar en cada elemento del menú, se mostrará un *Digimon* distinto.

La tarea

En primer lugar, desde la vista de carpetas a la izquierda **expande** *res* > *menu*. Haz *click* derecho en él y **selecciona** *New* > *Menu Resource File*.

Dale el nombre `bottom_menu.xml`. Verás que se autogenera un archivo así:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

</menu>
```

Dentro de este menú XML, **añade** un primer elemento `<item />`. Luego, **especifica** *tres* atributos: `android:id`, `android:title` y `android:icon`. **Así**:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/item1"
        android:title="Digimon 1"
        android:icon="@drawable/ic_monster" />

</menu>
```

¡Ese es el primer elemento del menú! Ahora, como queremos mostrar *tres Digimon* distintos, **añade** estos otros dos elementos:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/item1"
        android:title="Digimon 1"
        android:icon="@drawable/ic_monster" />
    <item
        android:id="@+id/item2"
        android:title="Digimon 2"
        android:icon="@drawable/ic_monster"/>
    <item
        android:id="@+id/item3"
        android:title="Digimon 3"
        android:icon="@drawable/ic_monster"/>
</menu>
```

Barra de navegación inferior

En `MonstersActivity` vamos a usar una `BottomNavigationView`, que nos permitirá ofrecer varias opciones al usuario desde la parte inferior de la pantalla.

¡Esto es muy útil! Cuando coges el teléfono con una mano, la parte inferior queda más cerca del pulgar que la parte superior. Es más *usable*.

Para empezar, **añade** un `<com.google.android.material.bottomnavigation.BottomNavigationView>` a `activity_monsters.xml` dentro del `<ConstraintLayout>`, al final, como se indica a continuación:

```
    <!--
    Hace falta disponer de la librería de Google Material para usar
BottomNavigationView.
    Para ello, basta con que en el archivo build.gradle se indique en
    'dependencies' la línea:
        implementation 'com.google.android.material:material:x.y.z'
    Con la versión x.y.z de la librería deseada.

    Esta línea YA está añadida en nuestro proyecto
    -->
    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottomNavigation"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"/>
```

(Como ves, las `constraints` están pensadas para que la barra se quede pegada abajo, ocupando todo el ancho)

Para completar lo necesario en `<com.google.android.material.bottomnavigation.BottomNavigationView>`, **añádele** el siguiente atributo `app:menu`, **así**:

```
        app:menu="@menu/bottom_menu"
```

De esta manera, nuestro `<BottomNavigationView>` ya está *ligado* al menú XML.

Un pequeño detalle...

También en `activity_monsters.xml`, para que imagen no se *superponga* a nuestra barra inferior, **posiciónala** adecuadamente:

```
    <ImageView
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:src="@drawable/togemon"
        android:contentDescription="Un Digimon poderoso"
        app:layout_constraintTop_toBottomOf="@id/monstersTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
-       app:layout_constraintBottom_toBottomOf="parent"
+       app:layout_constraintBottom_toTopOf="@id/bottomNavigation" />
```

¡Listo! Prueba a ver tu barra inferior en la `MonstersActivity`.

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

👍 0 👎 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 19, 2026, 10:35 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Respondiendo a clicks en la barra de navegación

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Añadiremos código a `MonstersActivity.java` para lanzar un `Toast` cada vez que se haga *click* en un elemento de la barra inferior

Descripción

Nuestra `BottomNavigationView` es bonita, pero no hace nada.

Vamos a aprender *cómo* responder a los *clicks* del usuario.

La tarea

Para empezar, en `MonstersActivity.java`, **instancia** una variable de tipo `BottomNavigationView` en el método `onCreate`, asociándola al elemento del XML, **así**:

```
public class MonstersActivity extends AppCompatActivity {

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_monsters);

        BottomNavigationView bar = findViewById(R.id.bottomNavigation);
        // ...
    }
}
```

Luego, sobre `bar`, **invoca** `setOnItemSelectedListener` (es similar a `setOnClickListener`):

```
        bar.setOnItemSelectedListener();
```

...y dentro de los paréntesis, **instancia** un *listener* de tipo `NavigationBarView.OnItemSelectedListener` *anónimo*.

Será anónimo. Cuando escribamos `new` *(y espacio en blanco)*, Android Studio nos permitirá autocompletar:

```
        bar.setOnItemSelectedListener(new
NavigationBarView.OnItemSelectedListener() {
            @Override
            public boolean onNavigationItemSelected(@NonNull MenuItem item)
{
                return false;
            }
        });
```

Como ves, sólo hay un método `onNavigationItemSelected`. En él, *recibimos* un parámetro tipo `MenuItem` que representa el *item* clicado por el usuario. Dentro de este `MenuItem` podemos usar `.getItemId()` para distinguirlo.

Escribe el siguiente código:

```
        bar.setOnItemSelectedListener(new
NavigationBarView.OnItemSelectedListener() {
            @Override
            public boolean onNavigationItemSelected(@NonNull MenuItem item)
{
                if (item.getItemId() == R.id.item1) {
                    // Hacer algo si el usuario pulsa Digimon 1
                }
                if (item.getItemId() == R.id.item2) {
                    // Hacer algo si el usuario pulsa Digimon 2
                }
                if (item.getItemId() == R.id.item3) {
                    // Hacer algo si el usuario pulsa Digimon 3
                }
                return false;
            }
        });
```

También debemos modificar `return false`. Con ese *booleano* devolvemos a Android si *hemos sido capaces de gestionar el click*. En la práctica, siempre vamos a poder, así que **sustitúyelo** por `true`:

```
-                return false;
+                return true;
```

Para terminar, **implementa** el siguiente comportamiento:

- Si el usuario pulsa en **Digimon 1**, se lanzará un *Toast* con el texto: `"Este es el primero"`
- Si el usuario pulsa en **Digimon 2**, se lanzará un *Toast* con el texto: `"Has clicado en Digimon 2"`
- Si el usuario pulsa en **Digimon 3**, se lanzará un *Toast* con el texto: `"Clicaste Digimon 3"`

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 19, 2026, 10:45 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Un primer fragmento

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Hablaremos de *Fragments* en Android
- Añadiremos un `FrameLayout` a `activity_monsters.xml` que servirá de contenedor para distintos *fragments*
- Crearemos un primer *Fragment*
- Se verá en el contenedor cuando el usuario pulse el primer ítem de la barra inferior

Descripción

Los *Fragments* son algo así como sub-*actividades* en Android. Representan una parte reutilizable de la interfaz de usuario y definen y administran su propio diseño. También tienen su propio ciclo de vida.

¡Trabajemos con *Fragments*!

La tarea

Como primer paso, **elimina** el `ImageView` de `activity_monsters.xml`:

```
-    <ImageView
-        android:layout_width="0dp"
-        android:layout_height="0dp"
-        android:src="@drawable/togemon"
-        android:contentDescription="Un Digimon poderoso"
-        app:layout_constraintEnd_toEndOf="parent"
-        app:layout_constraintStart_toStartOf="parent"
-        app:layout_constraintTop_toBottomOf="@id/monstersTitle"
-        app:layout_constraintBottom_toTopOf="@id/bottomNavigation"/>
```

En su lugar, **añade** un `<FrameLayout>`[1]. **Así**:

```
    <FrameLayout
        android:id="@+id/fragmentContainer"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toBottomOf="@id/monstersTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toTopOf="@id/bottomNavigation"
        />
```

Ahora que hemos sustituido nuestro `ImageView` por un `FrameLayout`, lo usaremos como *contenedor* de los *Fragments*.

Fragments

Tienen dos elementos: Interfaz XML y clase Java.

Interfaz XML

Haz click derecho en *res > layout* y **selecciona** *New > Layout Resource File*.

El **Root Element** será `androidx.constraintlayout.widget.ConstraintLayout`, como viene por defecto.

Dale el nombre `fragment_1.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

</androidx.constraintlayout.widget.ConstraintLayout>
```

Ahora, dentro de este `ConstraintLayout` **añade** la imagen que borramos anteriormente. **Así**:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:src="@drawable/togemon"
        android:contentDescription="Un Digimon poderoso" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

(Como ves, se usa `match_parent` para `layout_width` y `layout_height`. Esto hace que la imagen encaje (rellene) el padre).

Clase Java

Nuestro *Fragment* debe tener una clase asociada.

Haz click derecho en *src > com.afundacion.fps.library* y **selecciona** *New > Java Class*.

Dale el nombre `Fragment1`:

```
public class Fragment1 {

}
```

A continuación, **declara** que la clase *herede* de la superclase `Fragment`. **Así**:

```
import androidx.fragment.app.Fragment;

public class Fragment1 extends Fragment {

}
```

Además, estamos obligados a proveer un *constructor vacío*. Esta es una cuestión propia de cómo Android gestiona los *Fragments*. **Añádelo** así:

```
import androidx.fragment.app.Fragment;

public class Fragment1 extends Fragment {

    public Fragment1() {
    }
}
```

¡Ya casi hemos terminado con nuestro Fragment!

Sólo falta implementar el método `onCreateView`. Este método es análogo al `onCreate` de las actividades.

Teclea `onCreateView` debajo del constructor y verás que Android Studio nos permite autocompletar:

```
public class Fragment1 extends Fragment {

    public Fragment1() {
    }

    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable
ViewGroup container, @Nullable Bundle savedInstanceState) {
        return super.onCreateView(inflater, container, savedInstanceState);
    }
}
```

Para terminar este método, **reemplaza** la línea `return super.onCreateView` por el siguiente contenido:

```
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable
ViewGroup container, @Nullable Bundle savedInstanceState) {
        // Inflamos el XML fragment_1 y lo ponemos en el 'container'
        return inflater.inflate(R.layout.fragment_1, container, false);
    }
```

(Usamos el método `.inflate` del objeto provisto `inflater`. Este sirve para leer el archivo `fragment_1.xml` y convertirlo un objeto tipo `View` que Android podrá usar).

Uniendo barra inferior y *Fragments*

Ahora, en un par de líneas de código, vamos a unir los dos conceptos con los que hemos trabajado recientemente.

Para empezar, en `MonstersActivity.java`, **instancia** un nuevo `Fragment1` dentro del `if` del ejercicio anterior, correspondiente al código que se ejecuta cuando el usuario hace click en el primer elemento de la barra inferior:

```
import androidx.fragment.app.Fragment;

public class MonstersActivity extends AppCompatActivity {
    // ...

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        // ...
        bar.setOnItemSelectedListener(new
NavigationBarView.OnItemSelectedListener() {
            @Override
            public boolean onNavigationItemSelected(@NonNull MenuItem item)
{

                if (item.getItemId() == R.id.item1) {
                    // Aquí va el Toast de la tarea anterior. No la borres
                    // ...

                    // Instaciamos un Fragment1
                    Fragment myFragment = new Fragment1();
                }
                // Aquí van los if de item2 e item3
                // ...

            }
        });
    }
}
```

A continuación, con *una línea* **indica** que quieres *reemplazar* (visualmente) el *contenedor* con el *Fragment*.

Empieza así:

```
            Fragment myFragment = new Fragment1();
            getSupportFragmentManager()
```

...para obtener una *instancia* de un objeto *administrador* de *Fragments*, sobre el que indicarás que quieres empezar una nueva *transacción* **así**:

```
            Fragment myFragment = new Fragment1();
            getSupportFragmentManager().beginTransaction()
```

...y ahora sólo falta indicar que queremos reemplazar una vista con un `android:id` específico por el *Fragment* de la línea anterior. **Termina** así:

```
            Fragment myFragment = new Fragment1();
            // Recuerda que R.id.fragmentContainer es el id de nuestro
FrameLayout añadido al principio
getSupportFragmentManager().beginTransaction().replace(R.id.fragmentContain
er, myFragment)
```

¡Ah! Un último paso. **Haz** efectivo el cambio con `.commit()`:

```
            Fragment myFragment = new Fragment1();

getSupportFragmentManager().beginTransaction().replace(R.id.fragmentContain
er, myFragment).commit();
```

¡Ya has conseguido que se vea un Fragment cuando el usuario hace click en **Digimon 1** de la barra inferior!

Pero inicialmente `MonstersActivity` está vacía...

Cierto. Hemos eliminado el `<ImageView>` al principio así que era algo que cabía esperar.

Añade la siguiente línea al principio de `onCreate` en `MonstersActivity.java` para que, inicialmente, el *Fragment* sea visible:

```
public class MonstersActivity extends AppCompatActivity {
    // ...

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_monsters);

+       // Aquí, inicialmente también añadimos un Fragment1 a la interfaz
+
getSupportFragmentManager().beginTransaction().replace(R.id.fragmentContain
er, new Fragment1()).commit();

        // Otro código, como .setOnItemSelectedListener
        // ...
    }
}
```

¡Listo! Puedes verificar cómo funciona.

Por último

Sube tus cambios al repositorio en un nuevo commit.

---

1. Es un tipo de *layout* especial que se usa normalmente para albergar un *único hijo*. ↩

0 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

- **Rubén Montero** changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2
- **Rubén Montero** assigned to @pbecerof26 Oct 8, 2026, 12:57 PM GMT+2
- **Pablo Becerro Freire** closed Oct 20, 2026, 12:09 PM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Project
P PMDM
Pinned
Members
Work items 10
Branches
Merge requests 0
Pipelines
More features
Manage
Plan
Work items 10
Issue boards
Milestones
Wiki
Code
Build
Secure
Deploy
Operate
Monitor
Analyze
Settings

Un segundo fragmento

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

📚 Resumen

- Crearemos un segundo *Fragment*
- Se verá en cuando el usuario pulse el segundo *item* de la barra inferior

Descripción

Vamos a añadir un segundo *Fragment*, siguiendo los mismos pasos de la tarea anterior.

📦 La tarea

- **Añade** un XML `fragment_2.xml`. Contendrá un `ConstraintLayout`, y dentro un `ImageView`:
  - `contentDescription` será `"Otro Digimon muy poderoso"`.
  - `src` usará la imagen `garurumon`.
  - Rellenará por completo el `ConstraintLayout` (usa `match_parent`).
  - Tendrá el atributo `android:scaleType="centerCrop"`. Así aprenderemos sobre este atributo.
- **Añade** un `Fragment2.java`, análogo al de la tarea anterior.
- En `MonstersActivity.java`, cuando el usuario haga *click* en el *segundo item* del menú, **efectúa** una transacción que reemplace el contenedor por una *instancia* de `Fragment2` (análogo a la tarea anterior).

¡Verifica que puedes clicar en los dos primeros *items* de la barra inferior y funciona como es esperable!

🏆 Por último

Sube tus cambios al repositorio en un nuevo *commit*.

👍 0 👎 0 ☺ Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 20, 2026, 12:28 PM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment Reopen issue

Project
PMDM
Pinned
Members
Work items 10
Branches
Merge requests 0
Pipelines
More features
Manage
Plan
Work items 10
Issue boards
Milestones
Wiki
Code
Build
Secure
Deploy
Operate
Monitor
Analyze
Settings

Un tercer fragmento

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

Resumen

- Crearemos un tercer *Fragment*
- Se verá en cuando el usuario pulse el tercer *item* de la barra inferior

Descripción

Vamos a añadir un tercer *Fragment*.

La tarea

- **Añade** un XML `fragment_3.xml`. Contendrá un `ConstraintLayout`, y dentro un `ImageView`:
  - `contentDescription` será `"Un último Digimon muy fuerte"`.
  - `src` usará la imagen `birdramon`.
  - Rellenará por completo el `ConstraintLayout` (usa `match_parent`).
  - Tendrá el atributo `android:scaleType="fitXY"`. ¿Cómo se verá?
- **Añade** un `Fragment3.java`, análogo al de la tarea anterior.
- En `MonstersActivity.java`, cuando el usuario haga *click* en el *tercer item* del menú, **efectúa** una transacción que reemplace el contenedor por una *instancia* de `Fragment3`.

¡Verifica que puedes clicar todos los *items* de la barra inferior y funciona como es esperable!

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

👍 0 👎 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 22, 2026, 12:03 PM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment Reopen issue

Project
P PMDM
Pinned
Members
Work items 10
Branches
Merge requests 0
Pipelines
More features
Manage
Plan
Work items 10
Issue boards
Milestones
Wiki
Code
Build
Secure
Deploy
Operate
Monitor
Analyze
Settings

Un fragmento de nota... ¡o un 100% de ella!

Closed Issue created Oct 8, 2026, 12:57 PM GMT+2 by **Rubén Montero**

📚 Resumen

- Explicaremos el sistema de evaluación
- Ejecutaremos los *tests* de las tareas
- Si hay errores en nuestro código de las tareas anteriores, lo corregiremos

Descripción

La nota de cada *sprint* será calculada individualmente, lanzando *tests* automáticos sobre tu código.

- ¡Ojo! Cada tarea **no** vale lo mismo. Las tareas con más *tests* pesan más.

Lanzar los *tests*

Desde Android Studio, **expande** la carpeta:

```
com.afundacion.fp.library/ (androidTest)
```

...a la izquierda y **haz *click* derecho** en el primer *test*. Selecciona **Run '...'** y espera a que se compile la aplicación y se lance en el emulador.

Si se despliega una ventana inferior y ves *ticks verdes* abajo a la izquierda, entenderemos que nuestro código *pasa el test* de la tarea adecuadamente.

Ahora la misión es **verificar que todos los *tests* pasan** (es decir, *ticks verdes*).

Si encuentras que algún *test* no pasa (*tick rojo*), debes buscar el error y corregir tu código para que el *test* pase. Esa es nuestra misión. Podemos hacer los *commits* adicionales que queramos para arreglar errores en el código. ¡Nuestra nota será directamente proporcional al número de *tests* que pasen satisfactoriamente!

🏆 Por último

Una vez verifiques que **todos** los *tests* pasan correctamente... ¡Trabajo del *sprint* finalizado!

👍 0 👎 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 1 Oct 8, 2026, 12:57 PM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 8, 2026, 12:57 PM GMT+2

Pablo Beceiro Freire closed Oct 22, 2026, 12:59 PM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment Reopen issue

Project
PMDM
Pinned
Members
Work items 10
Branches
Merge requests 0
Pipelines
More features
Manage
Plan
Work items 10
Issue boards
Milestones
Wiki
Code
Build
Secure
Deploy
Operate
Monitor
Analyze
Settings

Petición de red

Closed Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

📚 Resumen

- Veremos cómo usar la librería Volley para lanzar una solicitud de red simple
- Si el servidor responde correctamente, mostraremos un *Toast*
- Haremos `git add`, `git commit` y `git push` para subir los cambios al repositorio

Descripción

Una aplicación móvil no es de mucha utilidad si no se conecta a Internet. Como mucho podremos programar una calculadora o un pequeño juego *offline*, pero, siendo realistas... ¿Cuántas *apps* instaladas en un dispositivo real acceden a Internet? ¡Todas!

HTTP

HiperText Transfer Protocol es el protocolo de aplicación más empleando en la red de redes: *Internet*.

Tu navegador intercambia mensajes HTTP con algún servidor cada vez que visitas una página web.

Nosotros, desde nuestra aplicación, lanzaremos una solicitud HTTP al servidor http://raspi y esperaremos una respuesta.

Usaremos una librería llamada **Volley**. Hace falta tener la línea:

```
implementation 'com.android.volley:volley:1.2.1'
```

...en `build.gradle`. *(Ya está añadido en nuestro proyecto)*

📦 La tarea

Como primer paso, **abre** un terminal (cmd.exe) en tu ordenador y cambia el directorio activo hasta la ubicación de tu repositorio usando `cd`. Luego, **haz**:

```
git pull
```

...para traer los cambios de remoto a local, que contienen el *esqueleto* del proyecto de este *sprint*.

Abre el proyecto `android-clips/android-frontend-clips` desde Android Studio.

¡Ojo! En este *sprint* tendremos dos carpetas principales:

- `android-frontend-clips/`: Es la principal. Contiene el proyecto Android y trabajaremos sobre ella.
- `django-backend/`: Contiene código que es parte del proyecto, pero del *backend*. No trabajaremos aquí, pero puedes echar un vistazo a medida que avancemos

Después de abrir el proyecto, desde Android Studio, **abre** *java* > *com.ajfundacion.fp.clips* > `MainActivity.java` que ya nos resulta familiar:

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Aquí lanzaremos una petición HTTP
    }
}
```

Lanzar una petición HTTP... ¿a dónde?

Al *endpoint* REST disponible en http://raspi:8000/health. Si abres dicha URL desde un navegador, verás una respuesta en formato JSON. La idea es solicitar ese JSON *desde Android*.

Como primer paso, **crea** una *instancia* de `JsonObjectRequest`[1]:

```
        // Aquí lanzaremos una petición HTTP
        JsonObjectRequest request = new JsonObjectRequest();
```

Ahora, entre los paréntesis de `JsonObjectRequest()`, debemos pasar cinco parámetros que recibe el método constructor:

1. Una *constante* indicando el método HTTP de la petición. Nosotros siempre usaremos `Request.Method.GET`.
2. Un *String* indicando la URL.
3. El cuerpo de la petición. Es opcional. Siempre usaremos `null`.
4. Un *listener* en el que escribiremos el código responsable de manejar una *respuesta de éxito*.
5. Un *listener* en el que escribiremos el código responsable de manejar una *respuesta de error*.

Para empezar, **escribe** los tres primeros parámetros:

```
        // Aquí lanzaremos una petición HTTP
        JsonObjectRequest request = new JsonObjectRequest(
                Request.Method.GET,
                Server.name + "/health",
                null,
                // ...
        );
```

Listeners

El *siguiente parámetro* del *constructor* es un `Response.Listener<JSONObject>`.

Si escribimos `new` y un *espacio en blanco*, Android Studio nos permitirá autocompletar y generar una *instancia anónima*. **Hazlo** así:

```
        // Aquí lanzaremos una petición HTTP
        JsonObjectRequest request = new JsonObjectRequest(
                Request.Method.GET,
                Server.name + "/health",
                null,
                new Response.Listener<JSONObject>() {
                    @Override
                    public void onResponse(JSONObject response) {

                    }
                }
        );
```

Aparecerá todo subrayado en *rojo*, porque el *constructor* recibe *cinco* parámetros, pero de momento sólo hemos indicado *cuatro*.

Para terminar, **añade** una coma (`,`) tras la última llave (`}`), y **escribe** `new`. Android Studio autocompletará generando un nuevo `Response.ErrorListener` *anónimo*:

```
        // Aquí lanzaremos una petición HTTP
        JsonObjectRequest request = new JsonObjectRequest(
                Request.Method.GET,
                Server.name + "/health",
                null,
                new Response.Listener<JSONObject>() {
                    @Override
                    public void onResponse(JSONObject response) {

                    }
                }, new Response.ErrorListener() {
            @Override
            public void onErrorResponse(VolleyError error) {

            }
        }
        );
```

¡*Perfecto*! Hemos construido un `JsonObjectRequest`. Ahora, *lo importante* es *qué* código ponemos en `onResponse` y `onErrorResponse`.

En `onResponse`, **muestra** un *Toast* que indique `"Reply OK"`:

```
                // ...
                new Response.Listener<JSONObject>() {
                    @Override
                    public void onResponse(JSONObject response) {
                        // Aquí mostramos un Toast
                        // que indique "Reply OK"
                    }
                }, new Response.ErrorListener() {
                // ...
```

Cola de peticiones de red

Añade un atributo privado a `MainActivity`, de tipo `RequestQueue`:

```
public class MainActivity extends AppCompatActivity {
    private RequestQueue queue;

    // ...
```

...y, en la tercera línea de `onCreate`, **inicializalo** empleando. **Así**:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
+       this.queue = Volley.newRequestQueue(context);
+
        // Aquí lanzaremos una petición HTTP
        // ...
```

Este *objeto* servirá para *lanzar* la petición `JsonObjectRequest` que hemos creado anteriormente. *Representa* una cola de peticiones de red. ¿Sólo hace falta invocar `queue.add`[2]

Hazlo *al final* de `onCreate`:

```
        // ...
        this.queue.add(request);
    }
```

Un último problemilla...

Si pruebas la aplicación, verás esta línea en los *logs*:

```
java.net.UnknownHostException: Unable to resolve host "raspi": No address
associated with hostname
```

Nuestro *emulador* no es capaz de traducir `raspi` a una dirección IP válida de la misma manera que nuestro ordenador anfitrión sí lo hace. La IP de `raspi` es asignada por el DHCP de la red, que el emulador *no alcanza* a ver. Si `raspi` fuera un nombre de dominio públicamente accesible (como `www.google.es`), no habría esta complicación.

Ante esta limitación del emulator en lo relativo al DNS, debemos sustituir `raspi` en `Server.java` por una IP válida `X.X.X.X`.

Sigue las instrucciones en `Server.java` para hacer un *ping*, y luego, **sustituye** `raspi` por la IP correcta.

Una vez hecho esto, prueba de nuevo.

¡Funciona!

¡Enhorabuena! Has lanzado tu primera petición de red.

🏁 Por último

Desde el terminal de Windows (cmd.exe), nos posicionamos en `android-clips/` y escribimos:

```
git add *
```

...para marcar los cambios del un nuevo *commit*. Después:

```
git commit -m "Tarea #20: Implementada petición simple a /health en
android-clips"
```

...y *por último* subimos los cambios al repositorio remoto:

```
git push
```

No está de más visitar la página de GitLab (https://raspi) y verificar que el *commit* se ha subido.

---

1. `JsonObjectRequest` es una clase de la librería Volley que *representa* una petición HTTP. Hay tres posibilidades: `StringRequest` (Petición sencilla que espera un *texto plano* como respuesta), `JsonObjectRequest` (Espera un objeto JSON (`{...}`) como respuesta) y `JsonArrayRequest` (Espera un *array* JSON (`[...]`) como respuesta). ↩
2. No, no falta sólo eso. Para que una aplicación pueda lanzar peticiones de red debe declara el permiso `INTERNET` en el archivo *manifest*. Además, si el tráfico va por `http` y no `https`, será necesario `usesCleartextTraffic="true"`. Puedes verificar que ambas cuestiones ya está en el `AndroidManifest.xml` del proyecto. ↩

0 0 Add design

Child items 0
Linked items 0

Activity

All activity Oldest first

- **Rubén Montero** changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2
- **Rubén Montero** assigned to @pbeceiro226 Oct 22, 2026, 9:53 AM GMT+2
- **Pablo Beceiro Freire** mentioned in commit 820a20c8 Oct 23, 2026, 1:09 PM GMT+2
- **Pablo Beceiro Freire** closed Oct 26, 2026, 9:47 AM GMT+1

Normal text

Write a comment or drag your files here...

Switch to plain text editing ⌘⇧M

Make this an internal note

Respuesta correcta del servidor

Closed Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

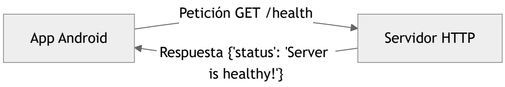
Resumen

- Veremos cómo trabajar con la respuesta del servidor a una petición
- Mostraremos en el *Toast* algo respondido por el servidor

Descripción

En la tarea anterior hemos aprendido a lanzar una petición HTTP.

Sabemos que esta petición es correspondida por nuestro servidor con una *respuesta* (en formato JSON).



Podemos verificarlo visitando desde el navegador el *endpoint* http://raspi:8000/health.

¿Y desde Android puedo usar esa respuesta?

¡Sí!

¿Cómo?

A través del `JSONObject` que obtenemos como *parámetro* en nuestro `onResponse`.

En `MainActivity.java`:

```
        // Aquí lanzaremos una petición HTTP
        JsonObjectRequest request = new JsonObjectRequest(
                Request.Method.GET,
                Server.name + "/health",
                null,
                new Response.Listener<JSONObject>() {
                    @Override
                    public void onResponse(JSONObject response) {
                        // Aquí el parámetro 'response' se corresponde
                        // con la RESPUESTA del SERVIDOR

                        // Lanzamos el Toast (tarea anterior)
                        // ...
```

Dado que la respuesta contiene una *clave-valor* con *clave* `status`, podríamos hacer:

```
// Imprime 'Server is healthy!'
System.out.println(response.getString("status"));
```

La tarea

Modifica el `Toast` de la tarea anterior, que mostraba `"Reply OK"`, para que ahora **concatene** *dos puntos* (`:`), un *espacio en blanco* y el *mensaje del servidor* (bajo la clase `"status"`):

```
- "Reply OK"
+ "Reply OK: " + response.getString("status")
```

- Si te encuentras con algún problema de una excepción *no controlada* al acceder[1] al JSON, basta con que la envuelvas en un `try-catch`.

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

---

1. Recuerda que esto sucede para obligarnos a considerar: *¿Qué pasa si la respuesta JSON no contiene la clave deseada? (En este caso, `status`)* ↩

👍 0 👎 0 Add design

Child items 0
Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Pablo Beceiro Freire closed Oct 26, 2026, 9:58 AM GMT+1

Normal text
Write a comment or drag your files here...
Switch to plain text editing
Make this an internal note
Comment Reopen issue

Help
Collapse sidebar

Respuesta incorrecta del servidor, o no

Closed Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Entenderemos cómo puede fallar una petición HTTP
- Mostraremos distintos *Toast* en distintos escenarios problemáticos

Descripción

En la tarea anterior hemos programado el *happy path* para mostrar un *Toast* con un mensaje del servidor. Pero es igual de importante programar los casos de error. Por ejemplo, ¿y si el usuario *no* tiene Internet?

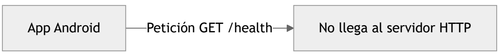
Con Volley, disponemos de `Response.ErrorListener`. Recuerda que lo habíamos añadido en la primera tarea:

```
        }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {

        }
```

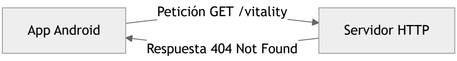
`onErrorResponse` es análogo a `onResponse`, pero para situaciones de error. Principalmente dos escenarios:

1) La conexión no se ha establecido



- El cliente no tiene conexión de red
- La red de destino no es accesible por cuestiones de *enrutamiento*
- El servidor HTTP recibe el mensaje, pero rechaza la conexión
- El servidor HTTP recibe el mensaje, pero lo ignora silenciosamente

2) La conexión funciona, pero el servidor responde un código de error



- El servidor HTTP acepta la conexión y responde con un código de error `4xx` ó `5xx`

Los *códigos de respuesta* HTTP se dividen en 5 familias:

1. `1xx`: Informacionales
2. `2xx`: De éxito
3. `3xx`: De redirección
4. `4xx`: De error de cliente
5. `5xx`: De error de servidor

La tarea

En `MainActivity.java`, **añade** el siguiente `if-else` dentro del `onErrorResponse`:

```
        @Override
        public void onErrorResponse(VolleyError error) {
            if (error.networkResponse == null) {
                // Error: No se ha establecido la conexión

            } else {
                // Error: El servidor ha dado una respuesta de error

                // La siguiente variable contendrá el código HTTP,
                // por ejemplo 401, 500,...
                int serverCode = error.networkResponse.statusCode;
            }
        }
```

(Como puedes observar, se estudia si `error.networkResponse` es nulo para distinguir entre los dos escenarios de error descritos anteriormente).

A continuación, dentro del `if`, **muestra** un `Toast` con el mensaje `"Problems with the connection!"`

Para terminar, dentro del `else`, **muestra** un `Toast` con el mensaje `"Server status: " + serverCode"`. Como ves, se concatena el código de error del servidor al *String*, para que dicho código se muestre al usuario en el `Toast`.

¡Enhorabuena! Has controlado por primera vez los errores que pueden surgir de una petición HTTP. Lanza la *app* y verifica tu código.

¿Cómo verificarlo?

1. Para verificar que *no* hay conexión, desde el emulador o un dispositivo físico apaga la Wi-Fi en *Ajustes* y apaga el Uso de Datos Móviles en *Ajustes*. ¡Ya no hay Internet!
2. Para verificar un *código de error* puedes, *momentáneamente*, alterar el *endpoint*:

```
-     Server.name + "/health",
+     Server.name + "/health2", // El servidor responderá 404 porque esta
URL no existe
```

(Luego déjalo como estaba)

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Pablo Beceiro Freire closed Oct 26, 2026, 10:05 AM GMT+1

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment Reopen issue

Una petición de una lista

Closed Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Lanzaremos una petición que solicita un *array* JSON
- Mostraremos un *SnackBar* si es exitosa

Descripción

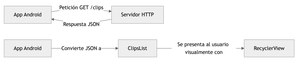
Nuestra aplicación lanza una petición de red que obtiene una respuesta sencilla.

```
{
  "status": "Server is healthy!"
}
```

Ahora vamos a lanzar una petición que obtendrá datos más elaborados. Un *array*:

```
[
  {
    "id": 1,
    "title": "Automatic Dialer (Part II)",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/automatic_dialer_part_ii.mp4"
  },
  {
    "id": 2,
    "title": "Desparasitándome",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/desparasitandome.mp4"
  },
  {
    "id": 3,
    "title": "Glasses",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/glasses.mp4"
  },
  {
    "id": 4,
    "title": "Marcador Automático",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/marcador_automatico.mp4"
  },
  {
    "id": 5,
    "title": "Tarta",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/tarta.mp4"
  }
]
```

El objetivo final es *presentar* estos datos al usuario.



En esta tarea, sólo vamos a preocuparnos de mandar la petición HTTP.

La tarea

Para empezar, en `MainActivity.java`, **crea** un método privado (dentro de la clase):

```
    private void requestClips() {

    }
```

...y luego, **invócalo** desde `onCreate`, como se indica a continuación:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        this.queue = Volley.newRequestQueue(context);

        // Solicitamos una lista de clips
        requestClips();

        // Aquí lanzaremos una petición HTTP
        // ...
```

(De esta manera, `requestClips` encapsulará la lógica para enviar la nueva petición HTTP y no haremos crecer excesivamente nuestro `onCreate`).

A continuación, dentro de **`requestClips`**, **instancia** un nuevo objeto `JsonArrayRequest`. Los parámetros enviados al método constructor serán:

- `Request.Method.GET`
- `Server.name + "/clips"`
- `null`
- `new Response.Listener<...>`
- `new Response.ErrorListener(...)`

Luego, **añade** dicha petición a la cola de red con `this.queue.add( ... );`

SnackBar

Para terminar la tarea, vamos a mostrar un `SnackBar` si la petición a `/clips` se ha completado correctamente. Es un componente similar al `Toast`, con estas diferencias:

- Puede permitir al usuario *clickar* para llevar a cabo alguna acción.
- Cuando lo construimos, debemos especificar una `View`. Se pintará *debajo* de dicha `View`.

Para mostrar un `SnackBar`, como primer paso, **añade** `id` al `ConstraintLayout` en `activity_main.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
+   android:id="@+id/main_layout"
    tools:context=".MainActivity">
```

...y luego, desde `MainActivity.java`, **añade** un atributo privado (`ConstraintLayout`) e **inicializalo** desde `onCreate` empleando `findViewById`. Quedará **así**:

```
public class MainActivity extends AppCompatActivity {
    private RequestQueue queue;
    private Context context = this;
    private ConstraintLayout mainLayout; // Añadimos un nuevo atributo

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        this.queue = Volley.newRequestQueue(context);
        this.mainLayout = findViewById(R.id.main_layout); // Lo asignamos con findViewById

        // ...
```

(En general es buena práctica almacenar los componentes como atributos privados e inicializarlos sólo una vez en `onCreate`. Esto se debe a que `findViewById` se considera una operación costosa).

Finalmente, otra vez en `MainActivity.java`, **añade** esta línea a tu nuevo `onResponse` (dentro de `requestClips`):

```
    Snackbar.make(mainLayout, "Clips obtained",
Snackbar.LENGTH_LONG).show();
```

¡Listo! Si pruebas la aplicación, deberías ver un `SnackBar` con el mensaje `"Clips obtained"` que demuestra que la petición a http://raspi:8000/clips se ha completado con éxito.

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

- **Rubén Montero** changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2
- **Rubén Montero** assigned to @pbecerof26 Oct 22, 2026, 9:53 AM GMT+2
- **Pablo Becerro Freire** closed Oct 26, 2026, 10:33 AM GMT+1

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment Reopen issue

Project
P PMDM
Pinned
Members
Work items 10
Branches
Merge requests 0
Pipelines
More features
Manage
Plan
Work items 10
Issue boards
Milestones
Wiki
Code
Build
Secure
Deploy
Operate
Monitor
Analyze
Settings

Una petición de una lista puede fallar

Closed Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Mostraremos un *SnackBar* si la petición de `/clips` falla

Descripción

Ya sabes que nuestra aplicación manda una petición HTTP a:

- http://raspi:8000/clips

...y recibe los datos. ¡Pero las cosas pueden ir mal!

¿Y si recibimos una respuesta inesperada, como un código `404`?

Debemos controlar, de forma similar a como hemos hecho en tareas anteriores.

La tarea

- Si hay un problema estableciendo la conexión al solicitar `/clips`, **muestra** un `SnackBar` con el mensaje `"Problems with the connection!"`.
- Si hay un código de error de servidor `XXX`, **muestra** en un `SnackBar` el mensaje: `"Server response XXX"`, donde `XXX` es el código de error.

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

👍 0 👎 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Pablo Beceiro Freire closed Oct 26, 2026, 10:49 AM GMT+1

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment Reopen issue

Help
Collapse sidebar

Mostrar el progreso

Closed Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Mostraremos una *ProgressBar* mientras la petición de `/clips` está activa

Descripción

Uno de los principios básicos creando aplicaciones Android es dar *feedback* visual al usuario. No puede ser que nuestra aplicación esté esperando por la respuesta de un servidor y el usuario no lo sepa. ¡Podría pensar que la *app* se ha congelado!

Un ProgressBar es un componente ideal para mostrar al usuario la idea de *algo está en progreso*.

Puede mostrar una barra que avanza, o bien un *círculo* que gira indefinidamente.

La tarea

Añade un `<ProgressBar>` en `activity_main.xml`:

```
    <ProgressBar
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
```

También, en dicho `<ProgressBar>`, **añade** los *atributos* necesarios para que:

- Esté centrado horizontalmente.
- Esté posicionado verticalmente como tú desees (arriba, en el centro, abajo...)
- Tenga un `android:id` (elígelo libremente)

A continuación, **añade** en `MainActivity.java` un atributo privado `ProgressBar` e **inicialízalo** en `onCreate` con `findViewById`, como se indica a continuación:

```
public class MainActivity extends AppCompatActivity {
    private RequestQueue queue;
    private Context context = this;
    private ConstraintLayout mainLayout;
+   private ProgressBar progressBar;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        this.queue = Volley.newRequestQueue(context);
        this.mainLayout = findViewById(R.id.main_layout);
+       this.progressBar = findViewById( /* El ID que tú hayas decidido */
);

        // Solicitamos una lista de clips
        // ...
```

`setVisibility`

Usaremos `.setVisibility` para mostrar u ocultar el `ProgressBar`.

Al principio del método `requestClips` en `MainActivity.java`, **añade** la línea `progressBar.setVisibility(View.VISIBLE)`:

```
    private void requestClips() {
        progressBar.setVisibility(View.VISIBLE);
        // ...
```

Y después, en `onResponse` y `onErrorResponse`, **invoca** nuevamente `setVisibility`, esta vez indicando `View.INVISIBLE`:

```
            @Override
            public void onResponse(JSONArray response) {
                progressBar.setVisibility(View.INVISIBLE);
                // ...
```

```
            @Override
            public void onErrorResponse(VolleyError error) {
                progressBar.setVisibility(View.INVISIBLE);
                // ...
```

¡Listo! Ahora tienes un `ProgressBar` que se muestra inicialmente cuando la *app* lanza una petición HTTP a http://raspi:8000/clips y se oculta cuando dicha petición se completa.

Si ejecutas la *app*, verás fugazmente aparecer el `ProgressBar`. También puedes configurar un *delay* de red en el emulador.

¿Y qué hay de la petición a `/health`?

La petición HTTP que configuramos en las tareas iniciales vamos a dejarla como está. Aunque sería una buena práctica, no vamos a preocuparnos de añadir un `ProgressBar` para esa petición.

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

👍 0 👎 0 Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Pablo Beceiro Freire closed Oct 29, 2026, 12:19 PM GMT+1

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Modelo

Closed · Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Crearemos una clase `Clip` que representa un objeto de la lista JSON enviada por el servidor
- Crearemos una clase `ClipsList`, que contiene varios `Clip` y representa la lista entera
- Declararemos un nuevo atributo de tipo `ClipsList` en nuestra `MainActivity`
- Lo inicializaremos con la respuesta del servidor

Descripción

Recordemos que los datos que vienen de:

- http://raspi:8000/clips

...son:

```
[
  {
    "id": 1,
    "title": "Automatic Dialer (Part II)",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/automatic_dialer_part_ii.mp4"
  },
  {
    "id": 2,
    "title": "Desparasitándome",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/desparasitandome.mp4"
  },
  {
    "id": 3,
    "title": "Glasses",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/glasses.mp4"
  },
  {
    "id": 4,
    "title": "Marcador Automático",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/marcador_automatico.mp4"
  },
  {
    "id": 5,
    "title": "Tarta",
    "videoUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/tarta.mp4"
  }
]
```

Consisten en una lista de *clips* que queremos mostrar al usuario.

Vamos a crear clases Java que representan los datos del servidor. Serán nuestras clases *modelo*. En concreto, trabajaremos con:

La tarea

Crea una nueva clase Java. **Dale** el nombre `Clip`.

Dentro de `Clip.java`, **añade** el siguiente código:

```
public class Clip {
    private int id;
    private String title;
    private String videoUrl;

    public Clip(JSONObject json) throws JSONException {
        this.id = json.getInt("id");
        this.title = json.getString("title");
        this.videoUrl = json.getString("videoUrl");
    }
}
```

(Como puedes ver, hay tres atributos que son los datos de un clip, y además, hemos añadido un método constructor especial que sirve para construir una instancia de `Clip` a partir de un `JSONObject`).

Para terminar con esta clase, **añade** los métodos ***getters*** de los tres atributos, con sus nombres estándar. Puedes autogenerarlos desde la barra superior de Android Studio con *Code > Generate > Getters and Setters*.

Luego, **crea** otra nueva clase Java. **Dale** el nombre `ClipsList`.

Dentro de `ClipsList.java`, **añade** el siguiente código:

```
public class ClipsList {
    private List<Clip> clips;

    public ClipsList(JSONArray array) {
        clips = new ArrayList<>();
        for (int i = 0; i < array.length(); i++) {
            try {
                JSONObject jsonElement = array.getJSONObject(i);
                Clip aClip = new Clip(jsonElement);
                clips.add(aClip);
            } catch (JSONException e) {
                throw new RuntimeException(e);
            }
        }
    }
}
```

(Como puedes observar, tiene un atributo `List<Clip>` que se inicializa a partir del contenido de un `JSONArray`, con el código del método constructor).

Para terminar con esta clase, también **añade** un método **getter** del único atributo que existe, con un nombre estándar (`getClips()`).

Usando `ClipsList`

Estas clases que hemos creado (`ClipsList.java` y `Clip.java`) son modelos. Representan datos que vienen del servidor, y gracias a ellas, podemos desentendernos de *parsear* el JSON del servidor y centrarnos en manejar los datos.

En `MainActivity.java`, **declara** un nuevo atributo de tipo `ClipsList`, y un *getter* y un *setter* como se indica a continuación:

```
public class MainActivity extends AppCompatActivity {
    // ...
    private ClipsList clips;

    // ...

    public void setClips(ClipsList clips) {
        this.clips = clips;
    }

    public ClipsList getClipsForTest() {
        return clips;
    }
```

(Como puedes ver, `getClipsForTest` no es un nombre estándar para un método getter. Este método sólo sirve para que funcione un test automático que verificarás al final del sprint).

Para terminar la tarea, en el `onResponse` de la petición a http://raspi:8000/clips, **añade** la siguiente línea `setClips(new ClipsList(response));`:

```
        // ...
        new Response.Listener<JSONArray>() {
            @Override
            public void onResponse(JSONArray response) {
                progressBar.setVisibility(View.INVISIBLE);
                Snackbar.make(mainLayout, "Clips obtained",
Snackbar.LENGTH_SHORT).show();

                // Parseamos la respuesta y la asignamos a nuestro atributo
                setClips(new ClipsList(response));
```

¡Listo! Aunque no hay cambios visuales en la aplicación, en esta tarea hemos conseguido almacenar como un atributo `ClipsList` los datos que vienen en la `response` del servidor.

¿Ha valido la pena?

Indudablemente. El patrón de arquitectura Modelo-Vista-Controlador es de los más populares y nosotros estamos poniéndolo en práctica.

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0

Linked items 0

Activity

All activity · Oldest first

- **Rubén Montero** changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2
- **Rubén Montero** assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Pablo Beceiro Freire closed Oct 29, 2026, 12:30 PM GMT+1

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment Reopen issue

RecyclerView

Closed Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Comprenderemos la idea esencial del componente visual `RecyclerView`
- Añadiremos un `<RecyclerView>` a `activity_main.xml`. De momento, no mostrará nada

Descripción

Mostrar una *lista* es relativamente común en la mayoría de aplicaciones: La lista de correos de la bandeja de entrada, lista de *posts* de Facebook, lista de noticias...

Un componente esencial en Android es RecyclerView. Análogo a UITableView de iOS, se dedica a *reciclar* las vistas asociadas a cada celda de la *lista*.

`RecyclerView`, ¿por qué?

Hoy en día existen 1025 pokemon.

Si quisiéramos desarrollar una actividad Android que los mostrase a todos, podríamos emplear una implementación demasiado sencilla del primitivo componente `ListView` en cuyo caso almacenaríamos en memoria RAM todos los datos.

Suponiendo que mostramos un PNG de 1024x1024 (~500Kb) por cada uno, necesitaremos:

500 Kb * 1025 = 512 500 Kb = 512,5 Mb

Esto es...

...más del doble de memoria RAM que tenía el primer dispositivo Android. ¡Demasiado!

`RecyclerView` reduce el uso de memoria RAM *reutilizando* (reciclando) las *celdas* visibles en pantalla.

Por ejemplo, si sólo caben 4 celdas en pantalla:

...la memoria RAM necesaria para tener las imágenes será como mucho de:

500 Kb * 4 = 2 000 Kb = 1,95 Mb

¡Vendido! Quiero un `RecyclerView`

Implementarlo llevará ciertos pasos:

1. Añadir un `<RecyclerView>` al XML de la actividad.
2. Añadir un nuevo fichero XML que represente una *celda*.
3. Añadir una clase Java vinculada al XML de la celda. Esta clase Java representa una *celda*, pero en código Java. Hereda de `ViewHolder`.
4. Añadir una clase *adaptador*. Esta clase heredará de `RecyclerView.Adapter`.

Para esta tarea, sólo añadiremos `<RecyclerView>` al XML.

La tarea

En `activity_main.xml`, **escribe** una nueva etiqueta `<RecyclerV`, y **dale** a ENTER para que Android Studio autocomplete:

```
<androidx.recyclerview.widget.RecyclerView
    android:layout_width=""
    android:layout_height="" 
```

Añade al `<androidx.recyclerview.widget.RecyclerView>` los atributos necesarios para que esté ajustado al contenedor, de tal forma que ocupe toda la pantalla. Puedes usar:

- `match_parent` en `layout_width` y `layout_height`
- `0dp` en `layout_width` y `layout_height`, y *constraints* `Start_toStart`, `End_toEnd`, `Top_toTop` y `Bottom_toBottom`

...lo que prefieras.

Para terminar, **añade** un `android:id` al `<RecyclerView>`. El que quieras. Lo usaremos más adelante.

(No te preocupes porque teóricamente el `<RecyclerView>` y el `<ProgressBar>` se solapen. Ambas vistas no estarán visibles simultáneamente).

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design

Child items 0
Linked items 0

Activity All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2
Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2
Pablo Beceiro Freire closed Oct 29, 2026, 12:50 PM GMT+1

Normal text
Write a comment or drag your files here...
Switch to plain text editing
Make this an internal note
Comment Reopen issue

Celda para nuestro RecyclerView (XML)

Closed Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Añadiremos un nuevo archivo `recycler_view_cell.xml`

Descripción

Aunque sabemos que hay *cinco* clips que vienen como respuesta a la petición http://raspi:8000/clips, la idea es que el servidor pueda responder *lo que le dé la gana* (una lista de 10, 20, 100 clips...) y nuestra *app* lo pinte correctamente. Es decir, nuestro `<RecyclerView>` muestra un número *indefinido* (en tiempo de compilación) de elementos.

Por ello, debemos añadir un único archivo de *layout* (XML) que representa una *celda*. Más adelante, será trabajo del *adaptador* encargarse de decidir *cuántas* celdas se pintan y con *qué* contenido.

En esta tarea, sólo definiremos la *maqueta* (*layout* XML) de una celda:

La tarea

En la jerarquía de archivos, desde *res* > *layout* **haz** *click* derecho > *New* > *Layout Resource File*. **Dale** el nombre `recycler_view_cell` y **haz** *click* en OK, manteniendo el `ConstraintLayout` como elemento raíz:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

</androidx.constraintlayout.widget.ConstraintLayout>
```

Modifica `layout_height` en `recycler_view_cell.xml` para que valga `wrap_content`:

```
 <?xml version="1.0" encoding="utf-8"?>
 <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
-   android:layout_height="match_parent">
+   android:layout_height="wrap_content">
+
</androidx.constraintlayout.widget.ConstraintLayout>
```

(Múltiples de estos `recycler_view_cell.xml` van a estar dentro de nuestro `<RecyclerView>`, que, recordemos, ocupa toda la pantalla. Por lo tanto, ¡no podemos dejar `match_parent` como altura! Si lo hacemos, cada celda llenará todo el alto de la pantalla).

Ahora, dentro de la celda, vamos a mostrar únicamente un *texto*. Será el *título* de un *clip* de vídeo devuelto por el API REST.

Añadiendo el contenido

Dentro del `<ConstraintLayout>`, **añade** un `<TextView>`. **Dale** un `android:id` libremente (el que quieras).

Posiciónalo como quieras dentro del `<ConstraintLayout>`, con los márgenes que consideres oportunos.

Para terminar, en dicho `<TextView>`, **indica** un tamaño de texto: `23sp`.

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

👍 0 👎 0 ☺ Add design

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Pablo Beceiro Freire closed Oct 30, 2026, 1:03 PM GMT+1

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Project
PMDM
Pinned
Members
Work items 10
Branches
Merge requests 0
Pipelines
More features
Manage
Plan
Work items 10
Issue boards
Milestones
Wiki
Code
Build
Secure
Deploy
Operate
Monitor
Analyze
Settings

Celda para nuestro RecyclerView (Java)

Open Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Añadiremos un nuevo archivo `ClipViewHolder.java`

Descripción

Estamos acostumbrados a escribir una clase Java cada vez que añadimos un fichero de interfaz XML.

- Cuando creamos una actividad, el XML lleva asociado una `Activity` en Java
- Cuando creamos un *fragment*, el XML lleva asociado un `Fragment` en Java

En esta ocasión, crearemos un `ViewHolder`. O sea, la clase Java para cada *celda* de nuestro `RecyclerView`.

La tarea

En la jerarquía de archivos, desde *java* > *com.afundacion.fp.clips* **haz** *click* derecho > *New* > *Java Class*. **Dale** el nombre `ClipViewHolder`:

```
package com.afundacion.fp.clips;

public class ClipViewHolder {
}
```

Como primer paso **indica** que `ClipViewHolder` *hereda* de `ViewHolder`. Concretamente, de `RecyclerView.ViewHolder`. **Así**:

```
package com.afundacion.fp.clips;

import androidx.recyclerview.widget.RecyclerView;

public class ClipViewHolder extends RecyclerView.ViewHolder {
}
```

Verás que aparece el error:

> There is no default constructor available in 'androidx.recyclerview.widget.RecyclerView.ViewHolder'

Esto se debe a que no está permitido *instanciar* una subclase de `ViewHolder` *directamente*. Es decir, por ser una subclase de `ViewHolder`, no podríamos hacer `v = new ClipViewHolder();`. ¡El constructor *vacío* ya no existe!

Estamos, por ello, obligados a crear un *constructor* que recibe una *vista* (`View`).

Usa la ayuda de Android Studio (bombillita roja) y **selecciona** *Create constructor matching super* para llegar a:

```
import android.view.View;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

public class ClipViewHolder extends RecyclerView.ViewHolder {
    public ClipViewHolder(@NonNull View itemView) {
        super(itemView);
    }
}
```

Luego, **añade** a `ClipViewHolder.java` un atributo privado de tipo `TextView`, llamado `clipTextView`. **Inicialízalo** desde el método constructor mediante `findViewById`, como se indica a continuación:

```
public class ClipViewHolder extends RecyclerView.ViewHolder {
    private TextView clipTextView;

    public ClipViewHolder(@NonNull View itemView) {
        super(itemView);
        clipTextView = itemView.findViewById( /* El ID que le diste a tu TextView en recycler_view_cell.xml */);
    }
}
```

(Como puedes ver, estamos invocando `findViewById` sobre `itemView`. Este objeto será creado dentro del adaptador en la siguiente tarea, donde uniremos todas las piezas).

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design Create merge request

Child items 0
Linked items 0

Activity All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Normal text
Write a comment or drag your files here...
Switch to plain text editing

Make this an internal note

Help
Collapse sidebar

Completando el RecyclerView

Open · Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

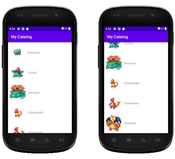
Resumen

- Añadiremos un nuevo archivo `ClipsAdapter.java`
- Entenderemos e implementaremos los tres métodos esenciales de un `RecyclerView.Adapter`
- Pondremos en marcha nuestro `RecyclerView` en `MainActivity.java`

Descripción

Como ya sabemos, un RecyclerView es un *componente* que presenta una lista de elementos al usuario, y recicla cíclicamente las celdas para ahorrar memoria RAM.

Así, el usuario puede hacer *scroll* en una cantidad indefinida de elementos:



Lo bueno es que nosotros no tenemos que implementar esa complicada *lógica de reciclaje*. Sólo tenemos que darle lo que ella necesita y hará el trabajo por nosotros.

¿Qué necesita?

Necesita la respuesta a estas tres preguntas:

1. *¿Cuántas celdas debo mostrar?*
2. *¿Cómo es (visualmente) cada celda?*
3. *¿Qué contenido le corresponde a cada celda?*

Vamos a darle esas respuestas mediante un *adaptador*. En concreto, una clase `RecyclerView.Adapter`.

La tarea

Crea una nueva clase Java y llámala `ClipsAdapter`. **Incluye** un atributo privado de tipo `ClipsList`, y un método constructor sencillo, donde asumimos que *alguien* le pasará al adaptador los datos del servidor. **Así**:

```
public class ClipsAdapter {
    private ClipsList clipsToBeShown;

    public ClipsAdapter(ClipsList clips) {
        this.clipsToBeShown = clips;
    }
}
```

A continuación, **añade** la cláusula de herencia (`extends`) a `ClipsAdapter.java`, como se indica a continuación:

```
import androidx.recyclerview.widget.RecyclerView;

public class ClipsAdapter extends RecyclerView.Adapter<ClipViewHolder> {
    private ClipsList clipsToBeShown;

    // ...
```

(Observa que incluimos `<ClipViewHolder>` en la declaración. ¡Nuestra clase! Este uso entre los símbolos "mayor que" y "menor que" `< >` se denomina genericidad, y es algo similar a usar `ArrayList<String>`).

Verás un error. **Usa** la ayuda de Android Studio para autogenerar los tres métodos obligatorios en un `RecyclerView.Adapter`.

`getItemCount`

- *¿Cuántas celdas debo mostrar?*

Completa este método devolviendo el tamaño del atributo `ClipsList` que añadimos al principio de la tarea. **Así**:

```
    @Override
    public int getItemCount() {
        return this.clipsToBeShown.getClips().size();
    }
```

`onCreateViewHolder`

- *¿Cómo es (visualmente) cada celda?*

Aquí toca *devolver* un `ViewHolder`. ¡Nuestro `ClipViewHolder`!

Añade este código al método `onCreateViewHolder`:

```
    @NonNull
    @Override
    public ClipViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
        // 1. Necesitamos un LayoutInflater. Lo creamos a partir de un
Context
        LayoutInflater inflater = LayoutInflater.from(parent.getContext());
        // 2. Con el LayoutInflater, 'inflamos' el XML y generamos una View
        View cellView = inflater.inflate(R.layout.recycler_view_cell,
parent, false);
        // 3. Esta View es la que pasamos al constructor de ClipViewHolder.
        //    ¡Y ya está lista!
        ClipViewHolder cellViewHolder = new ClipViewHolder(cellView);
        return cellViewHolder;
    }
```

(Como puedes ver, en este método "inflamos" manualmente la `recycler_view_cell.xml` y se la pasamos al constructor de `ClipViewHolder.java`, de la tarea anterior).

`onBindViewHolder`

- *¿Qué contenido le corresponde a cada celda?*

Este es el único método en el que no haremos un `return`. Simplemente, recibimos por parámetro:

- Un `ClipViewHolder`
- Un `int` indicando la posición

...y en el `ClipViewHolder` debemos *pintar* los datos asociados a esa posición.

Para empezar, cambia a la clase `ClipViewHolder.java`. Allí, **añade** este nuevo método, sustituyendo el comentario adecuadamente:

```
    public void bindClip(Clip clip) {
        this.clipTextView.setText(/* Usamos el getter de clip para obtener
el titulo */);
    }
```

¡Ya puedes regresar a `ClipsAdapter.java`! **Completa** `onBindViewHolder` *invocando* el método que acabas de crear. **Así**:

```
    @Override
    public void onBindViewHolder(@NonNull ClipViewHolder holder, int
position) {
        // Usamos .get(position) para acceder al 'enésimo' elemento de la
lista
        // O sea, el correspondiente a la posición 'position'
        Clip dataForThisCell =
this.clipsToBeShown.getClips().get(position);
        holder.bindClip(dataForThisCell);
    }
```

Hemos terminado nuestro trabajo en `ClipsAdapter.java`. Ahora falta darle uso.

Encajando las piezas

En `MainActivity.java`, **añade** un atributo de tipo `RecyclerView` e **inicializalo** mediante `findViewById` en `onCreate`:

```
public class MainActivity extends AppCompatActivity {
    //...
    private RecyclerView recyclerView;
    // ...

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // ...
        this.recyclerView = findViewById( /* Aquí, el ID que le diste a tu
<RecyclerView> en activity_main.xml */ );
        // ...
```

Luego, en el método `setClips` (que es donde trabajamos con los datos del servidor) **instancia** un `ClipsAdapter` y **vincúlalo** a tu `RecyclerView` invocando `.setAdapter`. **Así**:

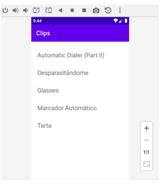
```
    public void setClips(ClipsList clips) {
        this.clips = clips;
        ClipsAdapter myAdapter = new ClipsAdapter(this.clips);
        recyclerView.setAdapter(myAdapter);
    }
```

Por último, **invoca** también `.setLayoutManager`[1] así:

```
    public void setClips(ClipsList clips) {
        this.clips = clips;
        ClipsAdapter myAdapter = new ClipsAdapter(this.clips);
        recyclerView.setAdapter(myAdapter);
+       recyclerView.setLayoutManager(new LinearLayoutManager(this));
    }
```

¡Enhorabuena! Has implementado tu primer `RecyclerView`. ¡Es un paso muy importante!

Según cómo hayas creado tu `recycler_view_cell.xml`, el resultado será *más o menos* como el siguiente:



Por último

Sube tus cambios al repositorio en un nuevo *commit*.

1. Un `RecyclerView` debe contar con un `LayoutManager`. Sirven para posicionar las celdas de maneras distintas. Nosotros, como queremos mostrar una lista (los elementos ocupan todo el ancho), empleamos `LinearLayoutManager`. *¿Podríamos mostrar una cuadrícula empleando un `GridLayoutManager`?* ↩

0 0 Add design Create merge request
Child items 0
Linked items 0
Activity All activity Oldest first
Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2
Rubén Montero assigned to @pbecerof26 Oct 22, 2026, 9:53 AM GMT+2
Normal text
Write a comment or drag your files here...
Switch to plain text editing
Help
Collapse sidebar

Clicks en las celdas

Open · Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Añadiremos un `OnClickListener` anónimo a cada celda `ClipViewHolder`
- Añadiremos también un atributo privado de tipo `Clip`
- Cuando se clique en una celda, se mostrará un *Toast* con información del Clip clicado

Descripción

Nuestro `RecyclerView` sirve para mostrar información de una lista de clips de video, obtenida de un API REST.

¡Vamos a permitir al usuario interaccionar con la lista!

Más concretamente, cuando el usuario clique una celda, se mostrará un `Toast` con el `id` del `Clip` correspondiente a esa celda.

Ese `id` en `ClipViewHolder.java`, sólo está disponible mediante el `Clip clip` del método:

```
public void bindClip(Clip clip) {
```

...pero, como verás a continuación, ¡necesitaremos almacenar el `Clip` y usarlo desde el método constructor!

Gráficamente

La tarea

Dentro de la clase `ClipViewHolder.java` tenemos un constructor que admite un parámetro tipo `View`. Es la vista de la celda, así que podemos usarla a nuestro antojo. **Establece** en dicho `itemView` un `OnClickListener` anónimo **así**:

```
// Método constructor
public ClipViewHolder(@NonNull View itemView) {
    // ...
    itemView.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {

        }
    });
```

Luego, **añade** a `ClipViewHolder.java` un atributo privado tipo `Clip`:

```
public class ClipViewHolder extends RecyclerView.ViewHolder {
    // Nuevo atributo
    private Clip clip;
    // ...
```

...e **inicializalo** añadiendo esta línea al método `bindClip`:

```
    public void bindClip(Clip clip) {
        //
        this.clip = clip;
```

Regresando al `OnClickListener` anónimo, **añade** las siguientes líneas para obtener el `id` del clip y conseguir un `context` que servirá para mostrar un `Toast`:

```
        itemView.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
+               int clipId = clip.getId();
+               Context context = view.getContext();
+
+               // Mostramos un Toast para completar la tarea
+               // ...

            }
        });
```

Finalmente, para completar el método `onClick` y la tarea muestra un `Toast` con el mensaje `"Clicked on ViewHolder with clipId: " + clipId`.

Por último

Sube tus cambios al repositorio en un nuevo commit.

0 · 0 · Add design · Create merge request

Child items 0

Linked items 0

Activity

All activity · Oldest first

- **Rubén Montero** changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2
- **Rubén Montero** assigned to @pbecerrof78 Oct 22, 2026, 9:53 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment · Close issue

Project
PMDM
Pinned
Members
Work items 10
Branches
Merge requests 0
Pipelines
More features
Manage
Plan
Work items 10
Issue boards
Milestones
Wiki
Code
Build
Secure
Deploy
Operate
Monitor
Analyze
Settings

Actividad que muestra vídeos (pero todavía no)

Open · Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Añadiremos una nueva actividad `VideoActivity`. De momento, sólo recibirá dos datos: `id` y `url` del video
- Los mostrará en un `Toast` (aún no reproducirá el video)

Descripción

Ya que podemos hacer click en cada celda y mostrar información asociada, pasaremos dicha información a una nueva actividad.

Hasta ahora no hemos *pasado datos de una actividad a otra*. ¡Pero se puede hacer!

Basta con usar `putExtra` cuando lanzamos la actividad y `getXXXExtra` en la nueva actividad.

Un detalle: Tipo de dato recuperado en la actividad *destino*

Mientras `putExtra` es el mismo método independientemente del tipo de dato (`int`, `String`, `boolean`,...) que queramos pasar, `getXXXExtra` tiene nombres distintos.

Debemos saber qué tipo de dato queremos recuperar y emplear el método apropiado[1]:

- `getIntExtra`
- `getLongExtra`
- `getFloatExtra`
- `getDoubleExtra`
- `getStringExtra`
- `getStringArrayExtra`
- `getStringArrayListExtra`
- ...

Otro detalle: Valores primitivos por defecto

Cuando usamos algún `getXXXExtra`, por ejemplo, `getIntExtra`, ponemos un segundo parámetro que representa el *valor por defecto*. En este ejemplo es `-1`:

```
// Si no se ha hecho `putExtra` de "NUMERO" desde la
// actividad origen, entonces aquí, en la actividad
// destino, el valor por defecto es -1
int elNumeroQueHanPasado = getIntExtra("NUMERO", -1);
```

Otro detalle más: Claves

Para identificar los valores que están siendo transmitidos de una actividad a otra se usan *claves* de tipo `String`, como `"NUMERO"` en el ejemplo anterior.

Podemos elegir cualquier `String` arbitrariamente. Siempre que coincidan en `.putExtra` y `.getXXXExtra`.

Si no coinciden... ¡La llamos! Por ello, es una buena práctica definir estas claves en un sólo sitio. Por ejemplo, como variables estáticas en la actividad[2][3]. Más adelante lo pondremos en práctica.

La tarea

Crea una nueva actividad con estas características:

- El archivo de interfaz se llamará `activity_video.xml` y sólo contendrá un `<ConstraintLayout/>` por defecto
- La clase Java será `VideoActivity.java`. Recuerda que debe heredar de `AppCompatActivity`
- ¡No olvides declarar la actividad en el *manifiesto*!

Luego, al final del `onClick` de `ClipViewHolder.java` **añade** estas líneas para empezar la nueva actividad:

```
            @Override
            public void onClick(View view) {
                int clipId = clip.getId();
                Context context = view.getContext();

                // Mostramos un Toast para completar la tarea
                // ...

+               // Y ahora... ¡Iniciamos la VideoActivity!
+               Intent intent = new Intent(context, VideoActivity.class);
+               context.startActivity(intent);
            }
```

A continuación, **añade** estas dos líneas entre medias de las dos anteriores. Sirven para indicar que queremos *transmitir* dos datos a la nueva actividad. **Sustituye** coherentemente el comentario de la URL del video por un valor correcto:

```
                // Y ahora... ¡Iniciamos la VideoActivity!
                Intent intent = new Intent(context, VideoActivity.class);
                intent.putExtra("CLIP_ID", clipId);
                intent.putExtra("CLIP_URL", /* Aquí, la URL del video. ¿De
qué atributo de la clase la podemos sacar? */);
                context.startActivity(intent);
```

Luego, ve a la `VideoActivity.java` que creaste en un principio.

Añade a `onCreate` las líneas que se muestran a continuación:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_video);

        Intent intent = getIntent(); // Con esto accedemos al Intent que se
usó para iniciar la actividad
        int clipId = intent.getIntExtra("CLIP_ID", -1);
        String clipUrl = intent.getStringExtra("CLIP_URL");
        Toast.makeText(this, "clipId: " + clipId,
Toast.LENGTH_LONG).show();
    }
```

(Recuerda que, originalmente, hemos obtenido estos datos del API REST).

¡Enhorabuena! Has conseguido pasar a la `VideoActivity` dos datos que provienen de la celda `ClipViewHolder`, y, además, puedes verificar que el `id` se muestra en un `Toast`.

Para terminar, vamos a refactorizar el código para aplicar una buena práctica aconsejada al principio.

Añade estos atributos estáticos a `VideoActivity.java`:

```
public class VideoActivity extends AppCompatActivity {
    public static final String INTENT_CLIP_ID = "CLIP_ID";
    public static final String INTENT_CLIP_URL = "CLIP_URL";
    // ...
```

Luego, **sustituyelos** en `onCreate` de `VideoActivity.java`:

```
-       int clipId = intent.getIntExtra("CLIP_ID", -1);
-       String clipUrl = intent.getStringExtra("CLIP_URL");
+       int clipId = intent.getIntExtra(VideoActivity.INTENT_CLIP_ID, -1);
+       String clipUrl =
intent.getStringExtra(VideoActivity.INTENT_CLIP_URL);
```

...y **sustituyelos** también en `ClipViewHolder.java`:

```
-               intent.putExtra("CLIP_ID", clipId);
-               intent.putExtra("CLIP_URL", /* ... */);
+               intent.putExtra(VideoActivity.INTENT_CLIP_ID, clipId);
+               intent.putExtra(VideoActivity.INTENT_CLIP_URL, /* ... */);
```

Así, las *claves* de los valores transmitidos están declaradas en un sólo sitio, y es más difícil que haya *bugs* por errores al teclear.

¡Tarea terminada!

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

1. Si queremos transmitir una instancia de una *clase* nuestra, deberá implementar la interfaz `Serializable`. ↩
2. El modificador `final` en Java establece que una variable no puede reescribirse. Es por tanto, constante ↩
3. Suele usarse notación distinta a lowerCamlCase cuando se definen **constantes** ↩

0 0
Add design
Create merge request
Child items 0
Linked items 0

Activity

All activity · Oldest first

- **Rubén Montero** changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2
- **Rubén Montero** assigned to @pbecerro126 Oct 22, 2026, 9:53 AM GMT+2

Normal text
Write a comment or drag your files here...
Switch to plain text editing
Make this an internal note
Comment
Close issue

¡Mostrando vídeos!

Open Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Añadiremos un `VideoView` a nuestra actividad `VideoActivity.java`
- Con la URL de vídeo que recibimos en la tarea anterior, reproduciremos un vídeo

Descripción

`VideoView` es una clase extremadamente útil en Android que permite reproducir un vídeo obtenido de una URL o de un *recurso* propio de la *app*.

La tarea

Añade un `<VideoView>` a `activity_video.xml`:

```
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <VideoView
        android:id="@+id/video_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

(Como ves, se usa `match_parent`. Esto deformará el vídeo para ajustarlo al contenedor. Nos vale, ya que esperamos que el vídeo tenga una relación de aspecto parecida a la pantalla. También es útil jugar con `wrap_content` y constraints según qué escenario).

Luego, en `VideoActivity.java`, **añade** un atributo tipo `VideoView` e **inicializalo** con `findViewById` en `onCreate`, como ya estamos acostumbrados:

```
public class VideoActivity extends AppCompatActivity {
    public static final String INTENT_CLIP_ID = "CLIP_ID";
    public static final String INTENT_CLIP_URL = "CLIP_URL";

    private VideoView videoView; // Nuevo atributo

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_video);
        Intent intent = getIntent();
        int clipId = intent.getIntExtra(VideoActivity.INTENT_CLIP_ID, -1);
        String clipUrl =
intent.getStringExtra(VideoActivity.INTENT_CLIP_URL);
        Toast.makeText(this, "clipId: " + clipId,
Toast.LENGTH_LONG).show();

        // VideoView
        videoView = findViewById(R.id.video_view);
```

A continuación, ¡basta con invocar `setVideoUri` para reproducir un vídeo de Internet asociado a una URL! **Hazlo** así:

```
        // VideoView
        videoView = findViewById(R.id.video_view);
        videoView.setVideoURI(Uri.parse(clipUrl));
        videoView.start();
```

Puedes lanzar la *app* y verificar que cualquier vídeo se ve correctamente. Quizás la pantalla del emulador se quede en negro bastante rato (incluso 4-5 minutos) si el ancho de banda de la red es muy reducido para descargar el vídeo. Sé paciente. ¡Seguramente verás tu vídeo!

Si giras la pantalla...

Es posible que el vídeo se vea demasiado *deformado* al girar la pantalla.

Vamos a optar por *deshabilitar* el cambio de orientación. Forzaremos a que la pantalla siempre se muestre *apaisada* (vista `landscape`).

En el *manifiesto*, **añade** `screenOrientation` a la etiqueta de nuestra *activity*. **Así**:

```
        <activity
            android:name=".VideoActivity"
            android:screenOrientation="landscape" />
```

Esto *obligará* a los usuarios a girar la pantalla cuando se inicie la actividad. ¡Pruébalo!

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0 Add design Create merge request

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Cuando termina, termina

Open Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

🧩 Resumen

- Terminaremos la `VideoActivity` cuando el vídeo termina

Descripción

Ahora mismo, nuestro *vídeo* llega al final y se queda parado. Ese es el comportamiento por defecto.

Vamos a hacer que la *actividad termine* (y, por tanto, regrese a la lista) cuando el vídeo llega al final.

Sólo necesitamos dos piezas:

`finish()`

Es un método de la clase `Activity`. Podemos invocarlo en cualquier sitio y terminará la actividad sobre la que se invoca. *Como* si el usuario pulsase *Atrás*.

`OnCompletionListener`

Un *listener* especial que podemos añadir a los `VideoView`. Responde a *que el vídeo termine de reproducirse*.

📦 La tarea

En primer lugar, **añade** un atributo `Activity` dentro de `VideoActivity.java`:

```
public class VideoActivity extends AppCompatActivity {

    // ...
    private Activity activity = this;
```

Luego, en el método `onCreate`, **invoca** `setOnCompletionListener()` sobre `videoView`.

Escribe `new` dentro de los paréntesis `( )` para que Android Studio autocomplete con un *listener anónimo*:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {

        // ...

        videoView.setOnCompletionListener(new
MediaPlayer.OnCompletionListener() {
            @Override
            public void onCompletion(MediaPlayer mediaPlayer) {

            }
        });
```

Dentro de `onCompletion`, **invoca** `activity.finish()` para que la actividad termine:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {

        // ...

        videoView.setOnCompletionListener(new
MediaPlayer.OnCompletionListener() {
            @Override
            public void onCompletion(MediaPlayer mediaPlayer) {
                activity.finish();
            }
        });
```

Prueba ahora a reproducir un vídeo.

¿Termina correctamente cuando llega al final?

🏆 Por último

Sube tus cambios al repositorio en un nuevo *commit*.

👍 0 👎 0 Add design Create merge request

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Help
Collapse sidebar

¿Quién aparece en la pantalla?

Open · Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

📚 Resumen

- Añadiremos `Character.java` y `CharactersList.java`, *modelos* para una nueva petición
- Añadiremos `CharacterViewHolder.java` y `character_recycler_cell.xml`, para un nuevo tipo de celda
- Añadiremos un `CharactersAdapter.java`, nuevo *adaptador* para un nuevo *RecyclerView*
- Aparecerá sobre el video cuando el usuario haga click, para mostrar los personajes que hay en la pantalla en ese instante

Descripción

En Amazon Prime Video y otras plataformas de *streaming*, puedes obtener información de qué personajes aparecen en la pantalla para un momento dado de un video.

Nosotros no vamos a ser menos.

Por suerte, nuestro API REST cuenta con una petición HTTP exclusivamente dedicada a eso. Sólo hay que enviar el `id` del clip de video, y el tiempo de reproducción (en milisegundos).

Por ejemplo:

- http://raspi:8000/clips/5/appearances?millisecond=16000

(En el clip `5`, a los `id` segundos, aparecen los siguientes personajes):

```
[
  {
    "name": "Lisa",
    "surname": "Simpson",
    "description": "Benevolente y educada hija de la familia Simpson",
    "imageUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/lisa.png"
  },
  {
    "name": "Bart",
    "surname": "Simpson",
    "description": "Pre-imaginito de la familia Simpson, siempre buscando problemas",
    "imageUrl":
"https://raw.githubusercontent.com/rubenmv0/fp/main/simpsons/bart.png"
  }
]
```

🥁 La tarea

Modelos: `Character.java` y `CharactersList.java`

Añade una nueva clase Java, `Character.java`. **Escribe** estos atributos:

```
    private String name;
    private String surname;
    private String characterDescription;
    private String imageUrl;
```

También, **implementa** los *getters* con su nomenclatura por defecto. Luego, **implementa** un método constructor que reciba un `JSONObject` por parámetro e inicialice los atributos de acuerdo a la respuesta de esquesta arriba (claves `"name"`, `"surname"`, `"description"` y `"imageUrl"`), de forma parecida a como has hecho en `Clip.java`.

Añade otra nueva clase Java, `CharactersList.java`. **Escribe** este atributo:

```
    private List<Character> characters;
```

También, **implementa** el *getter* con su nomenclatura por defecto. Luego, **implementa** un método constructor que reciba un `JSONArray` por parámetro e inicialice el atributo de acuerdo a la respuesta de esquesta arriba (esperando una lista de `Character`), de forma parecida a como has hecho en `ClipsList.java`.

Celda: XML y Java

Añade un nuevo archivo de interfaz `character_recycler_cell.xml`. **Haz** que el elemento raiz sea un `<ConstraintLayout>`, con `layout_width="match_parent"` y `layout_height="wrap_content"`. *Dentro* del `<ConstraintLayout>`, **añade** los dos elementos que se indican a continuación:

- Un `<ImageView>` *con id, anchura, altura y descripción de contenido* como se indica a continuación:

```
    <ImageView
        android:id="@+id/image_view_character"
        android:layout_width="64dp"
        android:layout_height="64dp"
        android:contentDescription="Personaje en clip"
        />
```

…y además, tendrá `scaleType="centerInside"` *y constraints para pegarlo a izquierda, derecha y arriba de su contenedor (pero no abajo!)*

- Un `<TextView>` *con id, anchura, altura, color de texto y tamaño de texto* como se indica a continuación:

```
    <TextView
        android:id="@+id/text_view_character"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textColor="#fff"
        android:textSize="14sp"
        />
```

…y además, tendrá constraints para pegarlo a izquierda, derecha y abajo de su contenedor. Por *arriba* estará pegado a la parte *inferior* de la imagen!.

Después, **añade** una nueva clase Java `CharacterViewHolder.java`. En su contenido, **incluye** el siguiente código:

```
public class CharacterViewHolder extends RecyclerView.ViewHolder {
    private TextView textView;
    private ImageView imageCharacter;


    public CharacterViewHolder(@NonNull View itemView) {
        super(itemView);
        textView = itemView.findViewById(R.id.text_view_character);
        imageCharacter = itemView.findViewById(R.id.image_view_character);
    }

    public void showData(Character character) {
        this.textView.setText(character.getName());
        Util.downloadBitmapToImageView(character.getImageUrl(),
this.imageCharacter);
    }
}
```

…y **añade** los `import` necesarios.

(Como puedes observar, esta clase Java es análoga a ClipViewHolder.java).

Adaptador

Añade una nueva clase Java `CharactersAdapter.java` que comience de esta manera:

```
public class CharactersAdapter extends
RecyclerView.Adapter<CharacterViewHolder> {
    private CharactersList charactersToShow;

    public CharactersAdapter(CharactersList charactersList) {
        this.charactersToShow = charactersList;
    }


    // Completa aquí los métodos de RecyclerView.Adapter
    /* ... */
}
```

…y **completa** su implementación. Será *análogo* a `ClipsAdapter.java` y contendrá `onCreateViewHolder`, `onBindViewHolder` y `getItemCount`.

Completar la *activity*

En `activity_video.xml`, **añade** una etiqueta `<RecyclerView>` a continuación del `<VideoView>` que ya existe. **Así**:

```
    <VideoView
        android:id="@+id/video_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

    <!-- Nuevo RecyclerView -->
    <!-- Salapa (por encima) el VideoView parcialmente. Es lo que queremos
-->
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recycler_view_characters"
        android:layout_width="75dp"
        android:layout_height="0dp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"/>
```

Y para dar uso a lo que hemos creado hasta ahora, abre `VideoActivity.java`.

En `VideoActivity.java` **añade** los nuevos atributos `recyclerView`, `queue` y `charactersOnScreen` que se muestran a continuación:

```
public class VideoActivity extends AppCompatActivity {
    /* ... */
    private RecyclerView recyclerView;
    private RequestQueue queue;
    private CharactersList charactersOnScreen;
```

En el método `onCreate`, **inicializa** `recyclerView` y `queue` como cabe esperar:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_video);

        queue = Volley.newRequestQueue(this); // Nueva cola. Idealmente, en
una app, se comparte una sola RequestQueue
        recyclerView = findViewById(R.id.recycler_view_characters); //
Buscamos por ID el recyclerView recién añadido

        // ...
```

Ahora, vamos a hacer que cuando el usuario pulse en el elemento principal de la pantall (`videoView`), la app mande una petición al API REST para consultar los personajes en pantalla.

Completa el código de `onCreate` como se muestra a continuación:

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {

        // ...

        videoView.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                // clipId fue inicializado más arriba, a través los
'Extras'
                // .getCurrentPosition() devuelve el milisegundo actual del
video
                sendAppearancesRequest(clipId,
videoView.getCurrentPosition());
            }
        });
    }
```

…y **añade** el método privado `sendAppearancesRequest`:

```
    private void sendAppearancesRequest(int clipId, int milliseconds) {

        /* ... */

    }
```

Dentro de este método:

- (1) **Crea** una nueva instancia de `JsonArrayRequest`
- (2) Para ello, **indica** el método HTTP `Request.Method.GET`
- (3) También, **indica** la URL apropiada que fue especificada al inicio de la tarea, en función de `clipId` y `milliseconds`. *(Recuerdo usar `Server.name` como host).*
- (4) Si la petición es exitosa, en `Response.Listener` **parsea** la respuesta del servidor y usa un método setter para almacenarla, *ondeingresante* a `MainActivity.java` (el método `setCharactersOnScreen` lo escribiremos a continuación!):

```
        new Response.Listener<JSONArray>() {
            @Override
            public void onResponse(JSONArray response) {
                CharactersList parsedServerResponse = new
CharactersList(response);
                setCharactersOnScreen(parsedServerResponse);
            }
        },
```

- (5) Si la petición falla, en `Response.ErrorListener` **muestra** un `Toast` con el mensaje que tú quieras.
- (6) Al final, tras instanciar tu `JsonArrayRequest`, **añade** dicha petición a la cola de red con `queue.add( /*...*/ );`.

Para finalizar la tarea, **escribe** el método setter `setCharactersOnScreen`, **así**:

```
    private void setCharactersOnScreen(CharactersList charactersOnScreen) {
        this.charactersOnScreen = charactersOnScreen;
        CharactersAdapter myAdapter = new
CharactersAdapter(this.charactersOnScreen);
        recyclerView.setAdapter(myAdapter);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
    }
```

(Funciona como el `setClips` de `MainActivity.java`. Esto vez puede ser `private`. No hace falta crear un getter).

Si has seguido todos los pasos de esta importante tarea, **¡enhorabuena!** Has implementado una importante funcionalidad haciendo uso de todo lo visto hasta ahora.

Adelante, lanza la *app* y después de esperar a que cargue algún video, toca la pantalla. *¿Qué sucede?*

⏳ Por último

Sube tus cambios al repositorio en un nuevo *commit*.

1. La *constraint* que necesitarás será `app:layout_constraintTop_toBottomOf="@id/image_view_character"` 📋

👍 0 👎 0 Add design Create merge request

Child items 0

Linked items 0

Activity

All activity Oldest first

- **Rubén Montero** changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2
- **Rubén Montero** assigned to @pbecerof26 Oct 22, 2026, 9:53 AM GMT+2

Normal text

Write a comment or drag your files here…

Switch to plain text editing

Fade Out

Open Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- En `VideoActivity.java`, añadiremos un `ObjectAnimator` que hará que la lista de personajes desaparezca con una animación
- Añadiremos un atributo *booleano* `isAnimating` a la actividad, para evitar efectos indeseados en la animación

Descripción

Nuestra funcionalidad de *lista de personajes en pantalla* es muy buena. ¡Vamos a mejorarla!

Ahora mismo, cuando pulsas en la pantalla, los personajes que aparecen permanecen ahí. Pero el vídeo avanza, así que deja de ser coherente...

Vamos a hacer que la lista sólo permanezca en la pantalla 3 segundos, y luego, desaparezca progresivamente durante 0,5 segundos.

Hay múltiples formas de llevar a cabo animaciones en Android. Nosotros vamos a usar el sencillo `ObjectAnimator`. Sirve para añadir animaciones que modifican valores de las propiedades.

La tarea

En `VideoActivity.java`, **añade** este código a `setCharactersOnScreen`:

```
    private void setCharactersOnScreen(CharactersList charactersOnScreen) {
        this.charactersOnScreen = charactersOnScreen;
        CharactersAdapter myAdapter = new
CharactersAdapter(this.charactersOnScreen);
        recyclerView.setAdapter(myAdapter);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));

        // Nuevo código
        ObjectAnimator fadeOut = ObjectAnimator.ofFloat(recyclerView,
"alpha", 1f, 0f);
        fadeOut.setDuration(500);
        fadeOut.setStartDelay(3000);
        fadeOut.start();
    }
```

¡Pruébalo! Verás que funciona bien... ¡sólo la primera vez!

Se debe a que el valor de transparencia (*alpha*) se transforma en `0`... ¡y ahí se queda! No vuelve a ser `1` hasta que el *delay* inicial de 3 segundos transcurre de nuevo.

Para asegurarnos que durante dicho *delay* el valor de *alpha* sea `1` (y por lo tanto, se vean los personajes), **establece** su valor manualmente al inicio:

```
+       recyclerView.setAlpha(1);
        ObjectAnimator fadeOut = ObjectAnimator.ofFloat(recyclerView,
"alpha", 1f, 0f);
        fadeOut.setDuration(500);
        fadeOut.setStartDelay(3000);
        fadeOut.start();
```

¡Funciona mejor! Aunque si pulsas repetidamente en la pantalla, el código se ejecuta una o varias veces *mientras* la animación toma lugar... ¡Y puede haber efectos impredecibles!

Para enfrentarnos al problema, podemos *deshabilitar* la animación si *ya está en curso*. De hecho, incluso nos ahorraremos la petición HTTP.

Añade un atributo *booleano* a la clase:

```
public class VideoActivity extends AppCompatActivity {
    /* ... */
    private boolean isAnimating = false;

    // ...
```

Luego, **establécelo** a `true` cuando la animación comienza, y a `false` cuando la animación termina. **Así**:

```
        recyclerView.setAlpha(1);
+       isAnimating = true; // Ponemos el booleano a true
        ObjectAnimator fadeOut = ObjectAnimator.ofFloat(recyclerView,
"alpha", 1f, 0f);
        fadeOut.setDuration(500);
        fadeOut.setStartDelay(3000);
        fadeOut.start();
        // Debemos usar un listener para saber cuándo termina la animación
        fadeOut.addListener(new AnimatorListenerAdapter() {
            @Override
            public void onAnimationEnd(Animator animation) {
+               // Y aquí, el booleano será false
+               isAnimating = false;
            }
        });
```

Con esto, tienes una variable `isAnimating` que vale `true` mientras el `RecyclerView` está en pantalla, y, cuando la animación termina (`alpha=0`), se vuelve `false`.

Añade la siguiente condición de guarda `if (isAnimating) { return; }` a `sendAppearancesRequest`:

```
    private void sendAppearancesRequest(int clipId, int milliseconds) {
        if (isAnimating) { return; }

        JsonArrayRequest /* ... */
```

¡Mejora completada! Ahora *no* se envía la petición HTTP si *ya se están mostrando* los personajes en la pantalla.

Verifica cómo funciona todo. ¿Estás satisfecho con el resultado?

Has desarrollado `Clips`, una *app* que pone en práctica los principios fundamentales de aplicaciones móviles multiplataforma. **¡Enhorabuena!**

Por último

Sube tus cambios al repositorio en un nuevo *commit*.

0 0
Add design Create merge request
Child items 0
Linked items 0
Activity All activity Oldest first
Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2
Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2
Normal text
Write a comment or drag your files here...
Switch to plain text editing
Make this an internal note
Help
Collapse sidebar

Project
P PMDM
Pinned
Members
Work items 10
Branches
Merge requests 0
Pipelines
More features
Manage
Plan
Work items 10
Issue boards
Milestones
Wiki
Code
Build
Secure
Deploy
Operate
Monitor
Analyze
Settings

Fade Bad Grades Out

Open Issue created Oct 22, 2026, 9:53 AM GMT+2 by **Rubén Montero**

Resumen

- Explicaremos el sistema de evaluación
- Ejecutaremos los *tests* de las tareas
- Si hay errores en nuestro código de las tareas anteriores, lo corregiremos

Descripción

La nota de cada *sprint* será calculada individualmente, lanzando *tests* automáticos sobre tu código.

- ¡Ojo! Cada tarea **no** vale lo mismo. Las tareas con más *tests* pesan más.

Lanzar los *tests*

Desde Android Studio, **expande** la carpeta:

```
com.afundacion.fp.clips/ (androidTest)
```

...a la izquierda y **haz *click* derecho** en el primer *test*. Selecciona **Run '...'** y espera a que se compile la aplicación y se lance en el emulador.

Si se despliega una ventana inferior y ves *ticks verdes* abajo a la izquierda, entenderemos que nuestro código *pasa el test* de la tarea adecuadamente.

Ahora la misión es **verificar que todos los *tests* pasan** (es decir, *ticks verdes*).

Si encuentras que algún *test* no pasa (*tick rojo*), debes buscar el error y corregir tu código para que el *test* pase. Esa es nuestra misión. Podemos hacer los *commits* adicionales que queramos para arreglar errores en el código. ¡Nuestra nota será directamente proporcional al número de *tests* que pasen satisfactoriamente!

Por último

Una vez verifiques que **todos** los *tests* pasan correctamente... ¡Trabajo del *sprint* finalizado!

👍 0 👎 0 Add design Create merge request

Child items 0

Linked items 0

Activity

All activity Oldest first

Rubén Montero changed milestone to %Sprint 2 Oct 22, 2026, 9:53 AM GMT+2

Rubén Montero assigned to @pbeceirof26 Oct 22, 2026, 9:53 AM GMT+2

Normal text

Write a comment or drag your files here...

Switch to plain text editing

Make this an internal note

Comment Close issue